Numerical and simulation techniques in finance

Nicolas Langrené

Beijing Normal-Hong Kong Baptist University

2025 Fall Semester

3 Finite difference methods

Solving a differential equation

The differential equation

$$\frac{\partial f}{\partial x} = -\frac{f(x)}{x}$$

is called an **ordinary differential equation (ODE)** because f takes only one input x. Solving an ODE means finding a function f which satisfies the differential equation. Here the solution is of the type

$$f(x) = \frac{C}{x}$$

where C is a constant.

Solving a differential equation

Indeed, the derivative of f is given by

$$\frac{\partial f}{\partial x} = -\frac{C}{x^2}$$

and therefore the ODE $\frac{\partial f}{\partial x} = -\frac{f(x)}{x}$ holds.

The solution $f(x) = \frac{C}{x}$ involves an arbitrary constant C. The function f is a solution of the ODE for any value of C. To find a particular solution, additional constraints are needed, such as imposing the solution to go through a predefined point. Such constraints are called **initial conditions**.

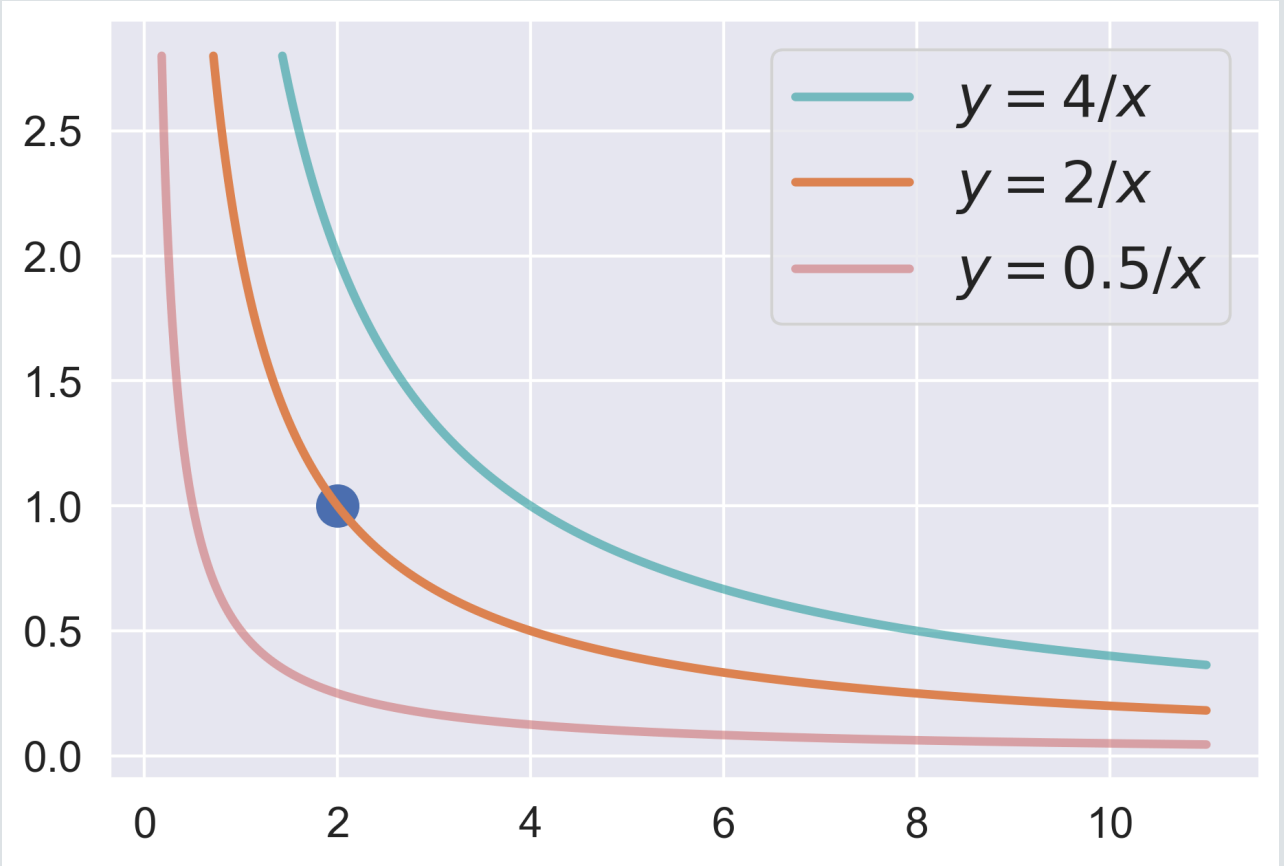
Solving an ODE with initial condition

Example: the initial condition

$$(x_0, y_0) = (2, 1)$$

implies that $C = 2$ and gives the particular solution

$$f(x) = 2/x$$



Solving a differential equation

There exist three main approaches to solve a differential equation:

▷ Finding an **analytical solution** (difficult, often does not exist).

▷ Looking for an **analytical approximation** (very difficult, approximation error tricky to assess).

▷ Compute a **numerical approximation** (applicable to any differential equation, easy to implement and analyze).

3.1 Finite differences

Forward-difference scheme

By approximating the first-order derivative $\frac{\partial f}{\partial x}$ by the forward-difference approximation $\frac{f(x+\Delta_x)-f(x)}{\Delta_x}$ with constant mesh size $0 < \Delta_x \ll |x|$, we can approximate the differential equation $\frac{\partial f}{\partial x} = -\frac{f(x)}{x}$ by

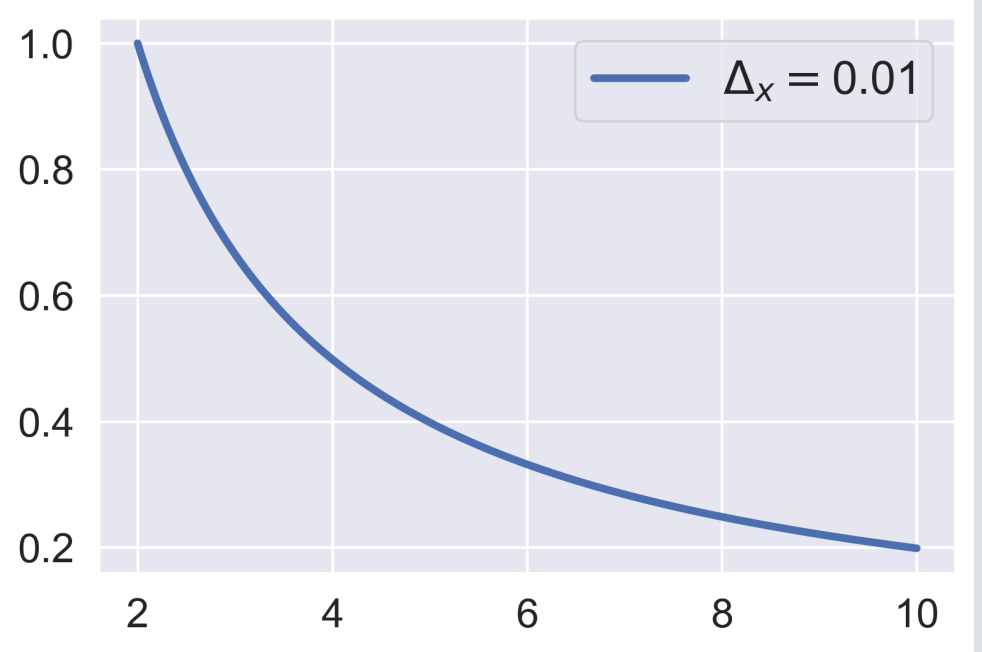
$$\frac{f(x+\Delta_x)-f(x)}{\Delta_x} = -\frac{f(x)}{x}$$

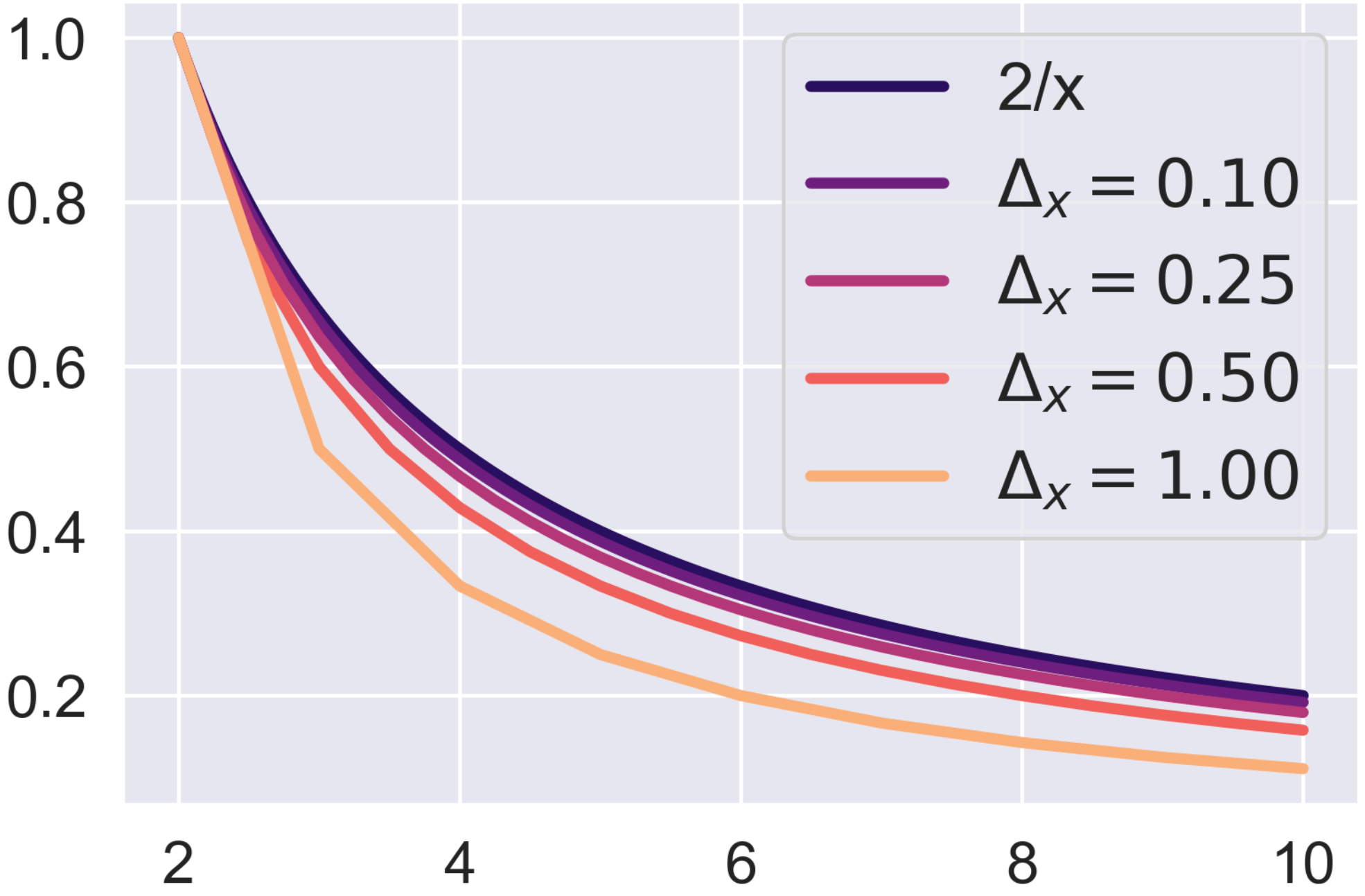
From where we get

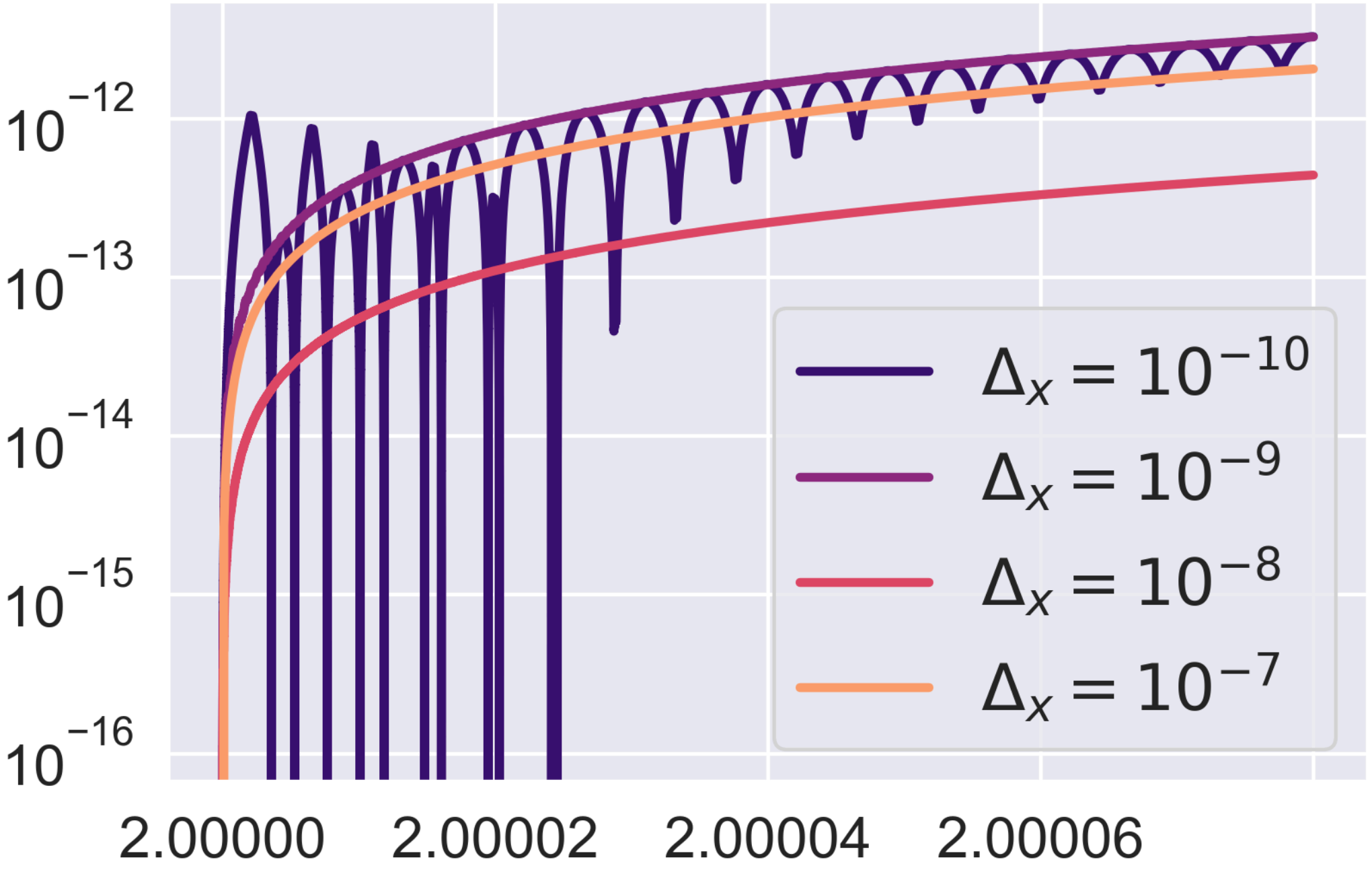
$$f(x+\Delta_x) = f(x)\left(1-\frac{\Delta_x}{x}\right)$$

Forward-difference scheme

```
import numpy as np
import seaborn as sns
(x0, xN) = (2.0, 10.0)
N = 801
x = np.linspace(x0,xN,N)
dx = x[1]-x[0]
fx = np.zeros_like(x)
fx[0] = 1.0
for (i,xi) in enumerate(x[:-1]):
    fx[i+1] = fx[i]*(1.0-dx/xi)
sns.lineplot(x=x,y=fx,label=
'$\Delta_x=$'+str(format(dx,'.9g'))
```




1.0
0.8
0.6
0.4
0.2
2
4
6
8
10
2/x
$\Delta_x = 0.10$
$\Delta_x = 0.25$
$\Delta_x = 0.50$
$\Delta_x = 1.00$


10^{-12}
10^{-13}
10^{-14}
10^{-15}
10^{-16}
2.00000
2.00002
2.00004
2.00006
$\Delta_x = 10^{-10}$
$\Delta_x = 10^{-9}$
$\Delta_x = 10^{-8}$
$\Delta_x = 10^{-7}$

Effect of the mesh size Δ_x

▷ The first picture shows that the smaller the mesh size Δ_x the closer the approximation to the true solution $f(x) = 2/x$. This is because the truncation error of the approximation of the first-order derivative is proportional to Δ_x.

▷ The second picture shows that the *absolute error* decreases until $\Delta_x \approx 10^{-8}$, below which it goes up again and numerical instabilities occur. This is because the float-rounding error becomes non-negligible for such small values of Δ_x, which deteriorates the approximation.

⚠ The smaller the mesh size, the more grid points and the higher the *computational cost*.

Centered-difference scheme

One can also approximate the first-order derivative $\frac{\partial f}{\partial x}$ by the centered-difference approximation $\frac{f(x+\Delta_x)-f(x-\Delta_x)}{2\Delta_x}$. The differential equation $\frac{\partial f}{\partial x} = -\frac{f(x)}{x}$ can be approximated

$$\frac{f(x+\Delta_x)-f(x-\Delta_x)}{2\Delta_x} = -\frac{f(x)}{x}$$

which can be rewritten as

$$f(x) = \frac{x}{2\Delta_x}\Big(f(x-\Delta_x) - f(x+\Delta_x)\Big)$$

Centered-difference scheme

In order to compute the equation

$$f(x) = \frac{x}{2\Delta_x}\Big(f(x-\Delta_x) - f(x+\Delta_x)\Big)$$

one needs to know *two points* $f(x-\Delta_x)$ and $f(x+\Delta_x)$. This means that this time we need to know *two initial conditions*.

Let us write the approximation for four successive points

$$(x_i, y_i)$$

where $x_i = x_0 + i \times \Delta_x$ and $y_i = f(x_i)\,, i = 1, 2, 3, 4.$

Centered-difference scheme

The finite-difference equation $y_i = \frac{x_i}{2\Delta_x}\Big(y_{i-1} - y_{i+1}\Big)$ yields

$$y_1 = x_1(y_0 - y_2)/(2\Delta_x)$$

$$y_2 = x_2(y_1 - y_3)/(2\Delta_x)$$

$$y_3 = x_3(y_2 - y_4)/(2\Delta_x)$$

$$y_4 = x_4(y_3 - y_5)/(2\Delta_x)$$

Finding the y vector amounts to solving a linear system.

Centered-difference scheme

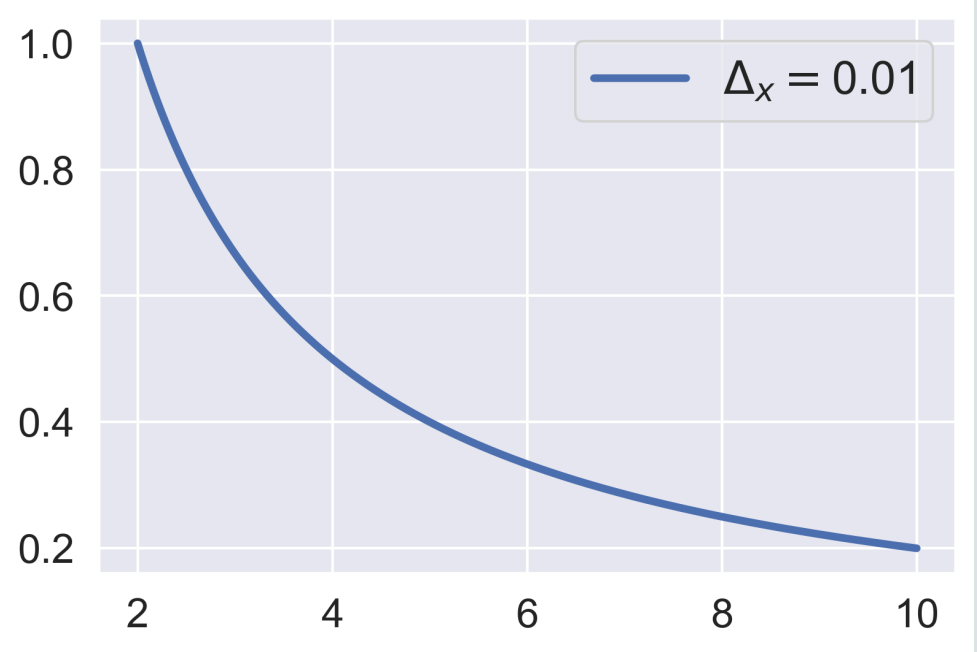
These four equations can be written as the following linear system

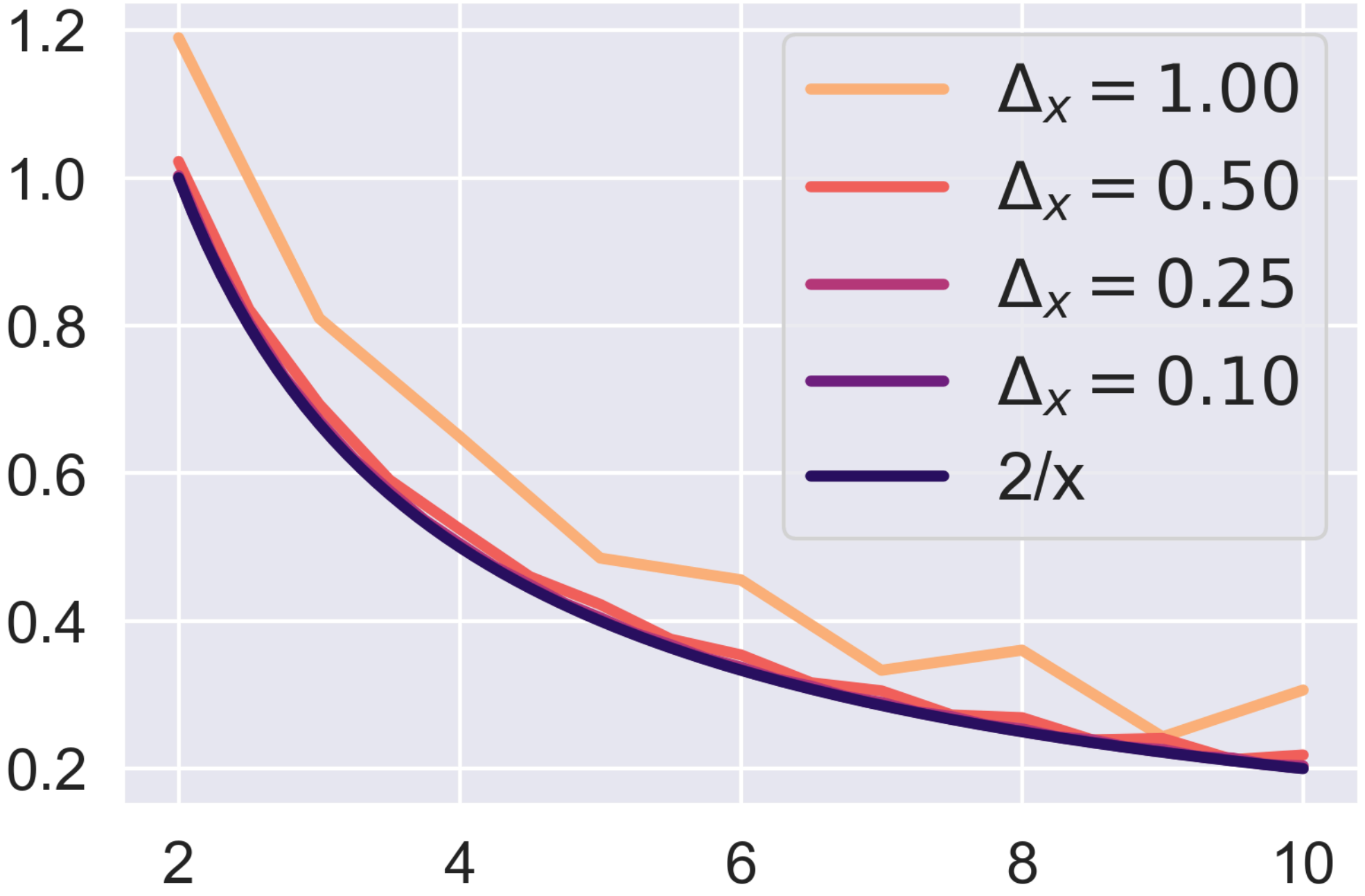
$$\begin{bmatrix} 1 & c_1 & & \\ -c_2 & 1 & c_2 & \\ & -c_3 & 1 & c_3 \\ & & -c_4 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} c_1 y_0 \\ 0 \\ 0 \\ -c_4 y_5 \end{bmatrix}$$

where $c_i = x_i/(2\Delta_x)$, which can be solve if we know the two initial conditions y_0 and y_5.

Centered-difference scheme

```
import numpy as np
import seaborn as sns
from scipy.sparse import diags
from scipy.sparse.linalg import spsolve
(x0,xN) = (2.0, 10.0)
N = 801
x = np.linspace(x0,xN,N)
dx = x[1]-x[0]
upper_diag = x[:-1]/(2.0*dx)
diag = np.ones_like(x)
lower_diag = -x[1:]/(2.0*dx)
diagonals = [lower_diag, diag, upper_diag]
A = diags(diagonals, [-1, 0, 1])
b = np.zeros_like(x)
b[0] = x[0]/(2.0*dx) * 2.0/(x0-dx)
b[-1] = -x[-1]/(2.0*dx) * 2.0/(xN+dx)
fx=spsolve(A,b) #np.linalg.solve(A.toarray(),b)
sns.lineplot(x=x,y=fx,label=
'$\Delta_x=$'+str(format(dx,'.9g')))
```




1.2
1.0
0.8
0.6
0.4
0.2
2
4
6
8
10
$\Delta_x = 1.00$
$\Delta_x = 0.50$
$\Delta_x = 0.25$
$\Delta_x = 0.10$
2/x

3.2 Classification of differential equations

Notations

Let u be a function of several variables. Depending on the context, u will be a function of two variables $u(t,x)$ or $u(x,y)$, or three variables $u(t,x,y)$. Here t is a time variable, while x and y are space variables. We use the following shorthand notation for partial derivatives:

$$u_t := \frac{\partial u}{\partial t}\ ,\quad u_x := \frac{\partial u}{\partial x}\ ,\quad u_y := \frac{\partial u}{\partial y}$$

$$u_{tt} := \frac{\partial^2 u}{\partial t^2}\ ,\quad u_{xx} := \frac{\partial^2 u}{\partial x^2}\ ,\quad u_{yy} := \frac{\partial^2 u}{\partial y^2}\ ,\quad u_{xy} := \frac{\partial^2 u}{\partial x \partial y}$$

Classification of differential equations

ODE (**O**rdinary **D**ifferential **E**quation): only one input variable x
PDE (**P**artial **D**ifferential **E**quation): multiple input variables $t, x, y, \ldots$

Classification criteria:

- Highest-order derivative
- **Linear** with **constant coefficients**

$$u_{xx} + 3u_{xy} + u_{yy} + u_x - u = e^{x-y}$$

- **Linear** with **variable coefficients**

$$\sin(xy)u_{xx} + 3x^2u_{xy} + u_{yy} + u_x - u = 0$$

Classification of differential equations

▷ **Non-linear**

$$u_{xx} + 3u_{xy} + u_{yy} + u_x^2 - u = e^{x-y}$$

▷ Two-dimensional second-order PDE

$$au_{xx} + bu_{xy} + cu_{yy} + du_x + eu_y + fu + g = 0$$

are classified based on their discriminant $\Delta = b^2 - 4ac$

▷ **hyperbolic** PDE if $\Delta > 0$

▷ **parabolic** PDE if $\Delta = 0$

▷ **elliptic** PDE if $\Delta < 0$

Classification of differential equations

- **Hyperbolic PDE** (temporal process). Ex: wave equation

$$u_{xx} - u_{tt} = 0$$

- **Parabolic PDE** (temporal propagation process). Ex: heat equation

$$u_t - u_{xx} = 0$$

- **Elliptic PDE** (equilibrium system). Ex: Laplace equation

$$u_{xx} + u_{yy} = 0$$

Classification of differential equations

A PDE problem is made of three components:

▷ The partial differential **equation**,

▷ The spatial and time **domains** in which the PDE has to hold,

▷ The **initial condition** (at time 0) and the **boundary conditions** (of the spatial domain).

Black-Scholes PDE

$$V_t + \frac{1}{2}\sigma^2 S^2 V_{SS} + (r - q)SV_S - rV = 0$$

$V = V(t, S)$ = option price
S = underlying asset price
σ = volatility of underlying asset price
r = risk-free interest rate
q = dividend yield

The Black-Scholes PDE is

- ▷ Linear with variable coefficients
- ▷ First-order in time, second-order in space
- ▷ Parabolic PDE with $b = c = 0$

Black-Scholes PDE

The initial condition (in time) is actually a *terminal condition* (backward PDE).

Terminal conditions and **boundary conditions** for **European call** option:

$$V(S,T) = (S-K)_+$$

$$V(0,t) = 0$$

$$\lim_{S\to\infty} V(S,t) = S - Ke^{-r(T-t)}$$

where K is the *strike price* defined in the European call option contract.

Black-Scholes PDE

Terminal conditions and **boundary conditions** for **European put** option:

$$V(S,T) = (K-S)_+$$

$$V(0,t) = Ke^{-r(T-t)}$$

$$\lim_{S\to\infty} V(S,t) = 0$$

See Chapters 4 and 6 of

[1] S. Shreve. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer, 2004.

3.3 The Black-Scholes partial differential equation

Why PDEs in finance?

If a real-valued stochastic process X satisfies the stochastic differential equation

$$dX_t = bX_t dt + \sigma X_t dW_t$$

with constant drift b, constant volatility σ, and W a standard Brownian motion (i.e. X is a Geometric Brownian motion), then the value function

$$V(t,x) = \mathbb{E}\left[\int_t^T e^{-r(s-t)} f(s, X_s) ds + e^{-r(T-t)} g(X_T) \middle| X_t = x\right]$$

Why PDEs in finance?

$V(t,x)$ is solution of the Black-Scholes PDE

$$rV(t,x) - V_t(t,x) - bxV_x(t,x) - \frac{1}{2}\sigma^2 x^2 V_{xx}(t,x) = f(t,x)\ ,\ \forall (t,x) \in [0,T) \times \mathbb{R}$$

$$V(T,x) = g(x)\ ,\ \forall x \in \mathbb{R}$$

This is a particular case of the **Feynman-Kac formula** ("Kac" Polish name pronounced as "cats").

The Feynman-Kac formula is a powerful tool which links conditional expectations and partial differential equations. It is the reason why many financial option pricing problems can be solved by a PDE approach.

General Feynman-Kac formula

If a stochastic process X valued in R^n satisfies the stochastic differential equation

$$dX_t^i = b_i(t, X_t)dt + \sum_{j=1}^{d} \sigma_{ij}(t, X_t)dW_t^j \ , \ 1 \leq i \leq n$$

where b is a drift vector function, W is a d-dimensional Brownian motion and σ is a $n \times d$ covariance matrix function. Then the value function

$$V(t,x) = \mathbb{E}\left[\int_t^T \exp\Big(-\int_t^s r(u, X_u)du \Big) f(s, X_s)ds + \exp\Big(-\int_t^T r(u, X_u)du \Big) g(X_T) \Big| X_t = x \right]$$

General Feynman-Kac formula

$V(t,x)$ is solution of the second-order PDE

$$r(t,x)V(t,x) - V_t(t,x) - \mathcal{L}V(t,x) = f(t,x)\ ,\ \forall (t,x) \in [0,T) \times \mathbb{R}^n$$

$$V(T,x) = g(x)\ ,\ \forall x \in \mathbb{R}^n$$

where the **infinitesimal generator**

$$\mathcal{L}V(t,x) := \lim_{h \to 0} \frac{\mathbb{E}\big[V(t+h, X_{t+h}) | X_t = x\big]}{h}$$

is

$$\mathcal{L}V(t,x) = b(t,x).\, V_x(t,x) + \frac{1}{2}\mathrm{tr}\big(\sigma(t,x)\sigma^\top(t,x)V_{xx}(t,x)\big)$$

General Feynman-Kac formula

It requires suitable regularity conditions for the functions b, σ, r, f, g, see [1] H. Pham. *Continuous-time stochastic control and optimization with financial applications*. Vol. 61. Stochastic Modelling and Applied Probability. 2009.

The Black-Scholes PDE is only **one particular case** of the Feynman-Kac PDE with $d = 1$, b and σ linear, r constant, $f = 0$ and g is the payoff of the European call/put option. It admits a closed-form solution, however generally the Feynman-Kac PDE **does not admit an explicit solution**, hence the need for numerical PDE methods (or Monte Carlo methods).

⚠ The PDE approach *does not apply* to models not built upon Brownian motions (which includes many interesting asset price models).

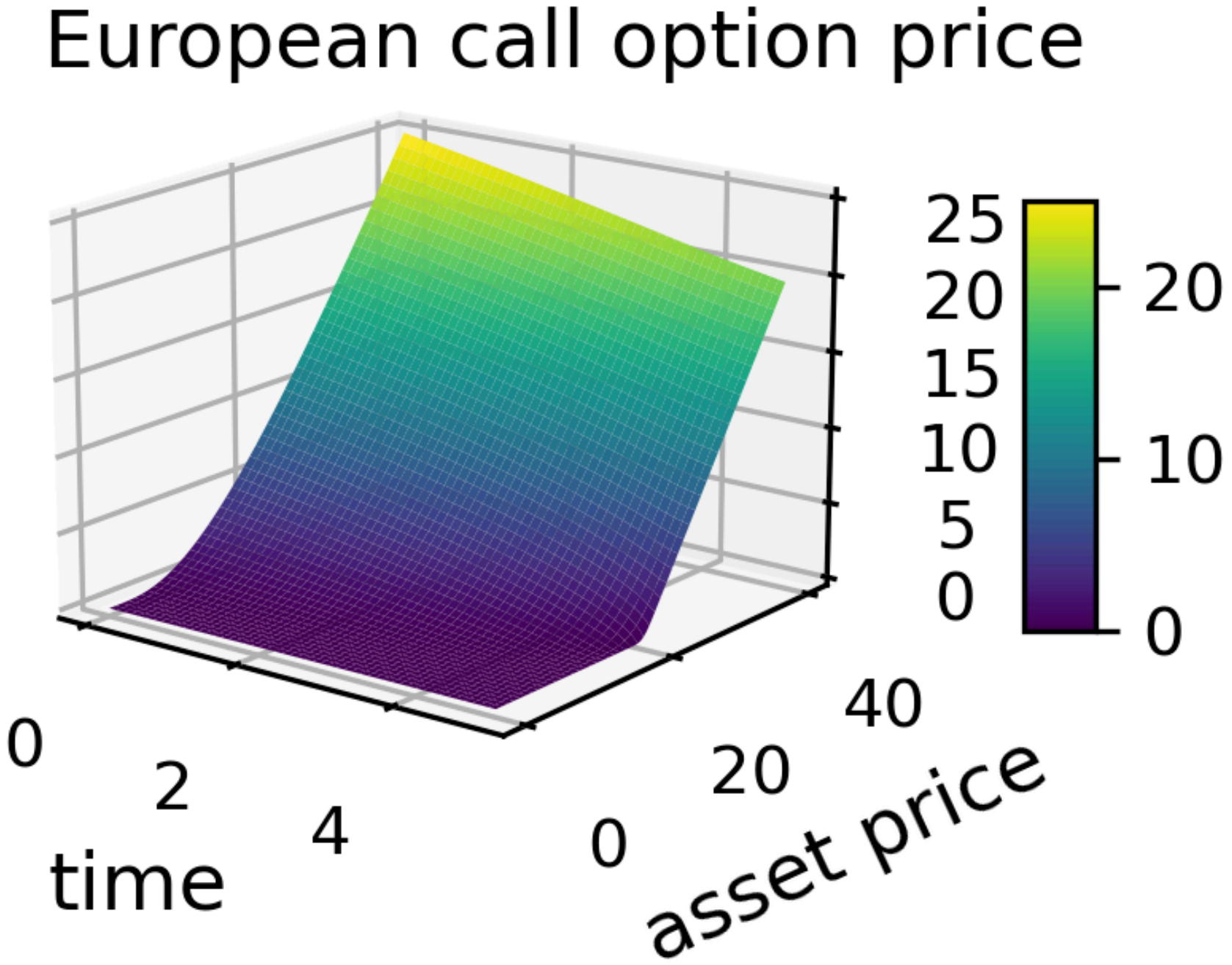
Back to the Black-Scholes PDE

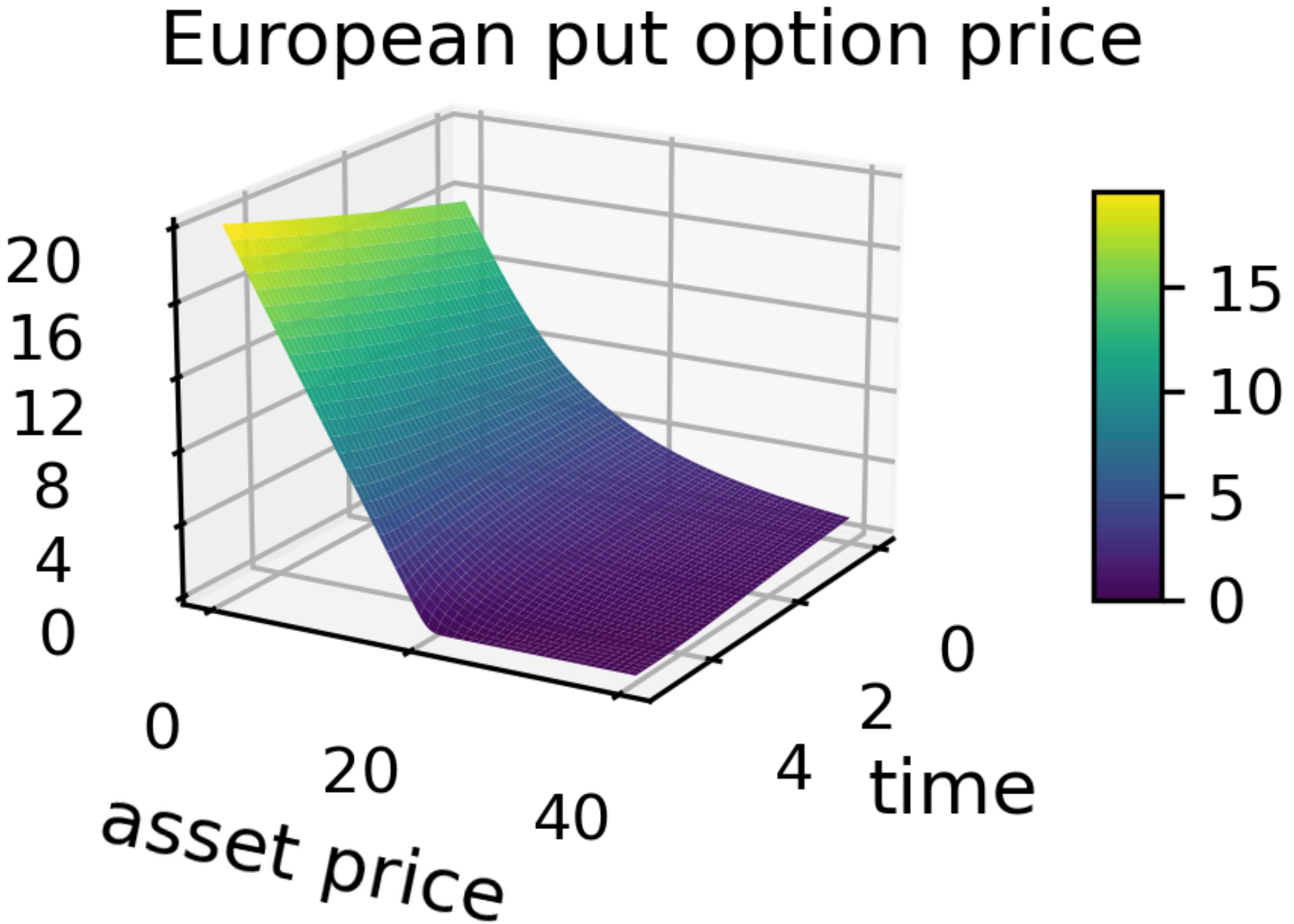
Setting $d = 1$, $r(t, x) = r$ constant, $b(t, x) = (r - q)x$, $\sigma(t, x) = \sigma x$, $f(t, x) = 0$ and $g(x)$ the terminal payoff, we retrieve the **Black-Scholes PDE**

$$V_t + \frac{1}{2}\sigma^2 S^2 V_{SS} + (r - q)SV_S - rV = 0$$

Which admits an explicit solution for European calls, European puts and a few other simple options (the **Black-Scholes formulas**).

In the rest of this subsection, we are going to use the Black-Scholes PDE to test numerical schemes for parabolic PDEs.


European call option price
25
20
15
10
5
0
20
10
0
0
2
4
time
0
20
40
asset price


European put option price
20
16
12
8
4
0
0
20
40
asset price
0
2
4
time
15
10
5
0

Finite-difference grid

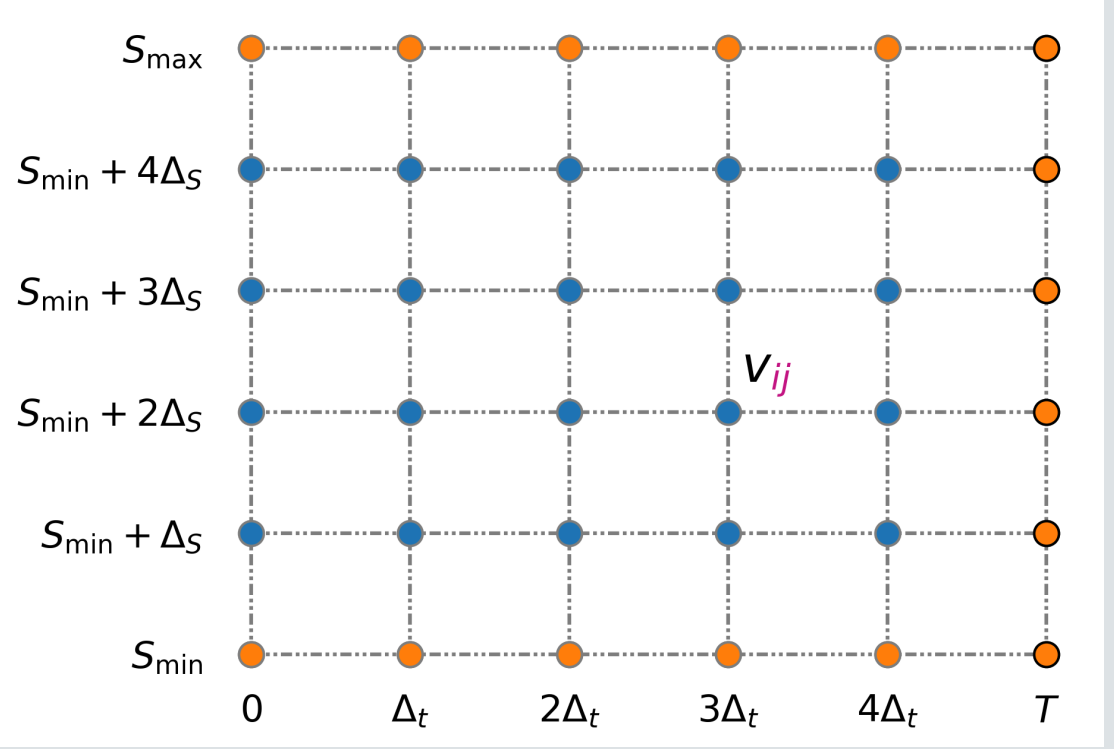
$$\Delta_S = (S_{\max} - S_{\min})/M$$
$$\Delta_t = T/N$$

The numerical approximation v_{ij} of $V(t, S)$ is only computed at the grid points (t_i, S_j)

$$(t_i, S_j) = (i\Delta_t, S_{\min} + j\Delta_S)$$

$$v_{ij} \approx V(i\Delta_t, S_{\min} + j\Delta_S)$$

It is an equality at the terminal condition points (T, S_j)



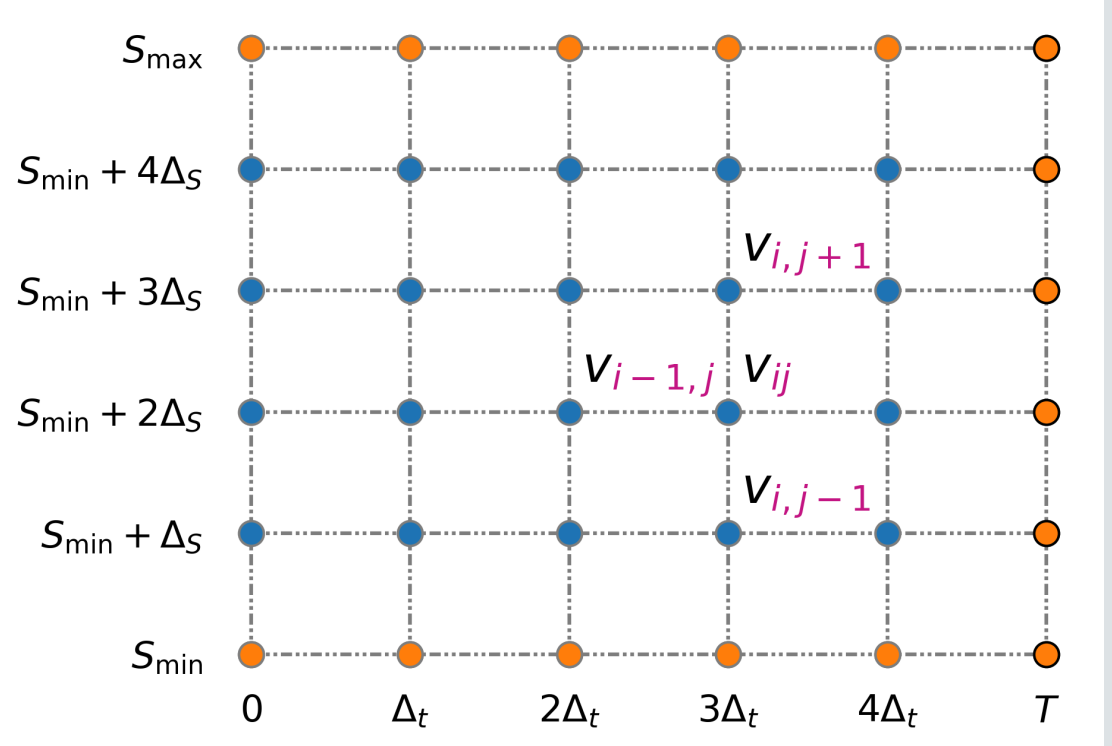
Finite-difference approximation

We approximate the partial derivatives V_t, V_S and V_{SS} by

$$V_t(t_i, S_j) \approx \frac{v_{ij} - v_{i-1,j}}{\Delta_t}$$

$$V_S(t_i, S_j) \approx \frac{v_{i,j+1} - v_{i,j-1}}{2\Delta_S}$$

$$V_{SS}(t_i, S_j) \approx \frac{v_{i,j+1} - 2v_{ij} + v_{i,j-1}}{\Delta_S^2}$$



Finite-difference scheme

We replace the partial derivatives in the Black-Scholes PDE by these finite-difference approximations

$$V_t + \frac{1}{2}\sigma^2 S^2 V_{SS} + (r-q)SV_S - rV = 0$$

$$\frac{v_{ij} - v_{i-1,j}}{\Delta_t} + \frac{1}{2}\sigma^2 S_j^2 \frac{v_{i,j+1} - 2v_{ij} + v_{i,j-1}}{\Delta_S^2} + (r-q)S_j \frac{v_{i,j+1} - v_{i,j-1}}{2\Delta_S} - rv_{ij} = 0$$

Finite-difference scheme

Define $a_j = \frac{\Delta_t \sigma^2 S_j^2}{2\Delta_S^2}$ and $b_j = \frac{\Delta_t (r-q) S_j}{2\Delta_S}$. For $j = 1, \ldots, M-1$:

$$v_{ij} - v_{i-1,j} + a_j(v_{i,j+1} - 2v_{ij} + v_{i,j-1}) + b_j(v_{i,j+1} - v_{i,j-1}) - r\Delta_t v_{ij} = 0$$

Grouping similar terms together, we obtain

$$v_{ij} - v_{i-1,j} + d_j v_{i,j-1} + m_j v_{ij} + u_j v_{i,j+1} = 0$$

where $d_j := a_j - b_j$, $m_j := -2a_j - r\Delta_t$ and $u_j := a_j + b_j$

System of finite-difference equation

$$v_{ij} - v_{i-1,j} + d_j v_{i,j-1} + m_j v_{ij} + u_j v_{i,j+1} = 0$$

The $j = 1, \ldots, M-1$ equations at time t_i are

$$\begin{array}{lllllllllll}
v_{i,1} & - & v_{i-1,1} & + & d_1 v_{i,0} & + & m_1 v_{i,1} & + & u_1 v_{i,2} & = & 0 \\
v_{i,2} & - & v_{i-1,2} & + & d_2 v_{i,1} & + & m_2 v_{i,2} & + & u_2 v_{i,3} & = & 0 \\
 & \vdots & & \vdots & & \vdots & & \vdots & & \vdots & \\
v_{i,j} & - & v_{i-1,j} & + & d_j v_{i,j-1} & + & m_j v_{i,j} & + & u_j v_{i,j+1} & = & 0 \\
 & \vdots & & \vdots & & \vdots & & \vdots & & \vdots & \\
v_{i,M-1} & - & v_{i-1,M-1} & + & d_{M-1} v_{i,M-2} & + & m_{M-1} v_{i,M-1} & + & u_{M-1} v_{i,M} & = & 0
\end{array}$$

System of finite-difference equation

In matrix notation, the system of equation is

$$\mathbf{v}_{i,1:M-1} - \mathbf{v}_{i-1,1:M-1} + \mathbf{P}\mathbf{v}_{i,0:M} = 0$$

where the $(M-1) \times (M+1)$ matrix $\mathbf{P}$ is defined by

$$\mathbf{P} = \begin{bmatrix} d_1 & m_1 & u_1 & 0 & \cdots & \cdots & 0 \\ 0 & d_2 & m_2 & u_2 & & & \vdots \\ \vdots & & d_3 & \ddots & \ddots & & \vdots \\ \vdots & & & \ddots & \ddots & u_{M-2} & 0 \\ 0 & \cdots & \cdots & 0 & d_{M-1} & m_{M-1} & u_{M-1} \end{bmatrix}$$

System of finite-difference equation

We can decompose the set of rows $\bar{\Omega} := \{0, 1, 2, \ldots, M\}$ into the interior rows $\Omega := \{1, 2, \ldots, M-1\}$ and the boundary rows $\partial\Omega := \{0, M\}$.
We introduce the vectors

$$\mathbf{v}_i^{\bar{\Omega}} = \begin{bmatrix} v_{i,0} \\ v_{i,1} \\ v_{i,2} \\ \vdots \\ v_{i,M-1} \\ v_{i,M} \end{bmatrix} \in \mathbb{R}^{M+1}, \quad \mathbf{v}_i^{\Omega} = \begin{bmatrix} v_{i,1} \\ v_{i,2} \\ \vdots \\ v_{i,M-1} \end{bmatrix} \in \mathbb{R}^{M-1}, \quad \mathbf{v}_i^{\partial\Omega} = \begin{bmatrix} v_{i,0} \\ v_{i,M} \end{bmatrix} \in \mathbb{R}^2$$

System of finite-difference equation

We introduce the matrices $\mathbf{A} \in \mathbb{R}^{(M-1)\times(M-1)}$ and $\mathbf{B} \in \mathbb{R}^{(M-1)\times 2}$

$$\mathbf{A} = \begin{bmatrix} m_1 & u_1 & 0 & \cdots & \cdots \\ d_2 & m_2 & u_2 & & \\ & d_3 & \ddots & \ddots & \\ & & \ddots & \ddots & u_{M-2} \\ \cdots & \cdots & 0 & d_{M-1} & m_{M-1} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} d_1 & 0 \\ 0 & \vdots \\ \vdots & \vdots \\ \vdots & 0 \\ 0 & u_{M-1} \end{bmatrix}$$

such that $\mathbf{P} = [\mathbf{B}[:,0] \;\; \mathbf{A} \;\; \mathbf{B}[:,1]]$ and therefore

$$\mathbf{P}\mathbf{v}_i^{\bar{\Omega}} = \mathbf{A}\mathbf{v}_i^{\Omega} + \mathbf{B}\mathbf{v}_i^{\partial\Omega}$$

Explicit scheme

The PDE scheme with previous notation

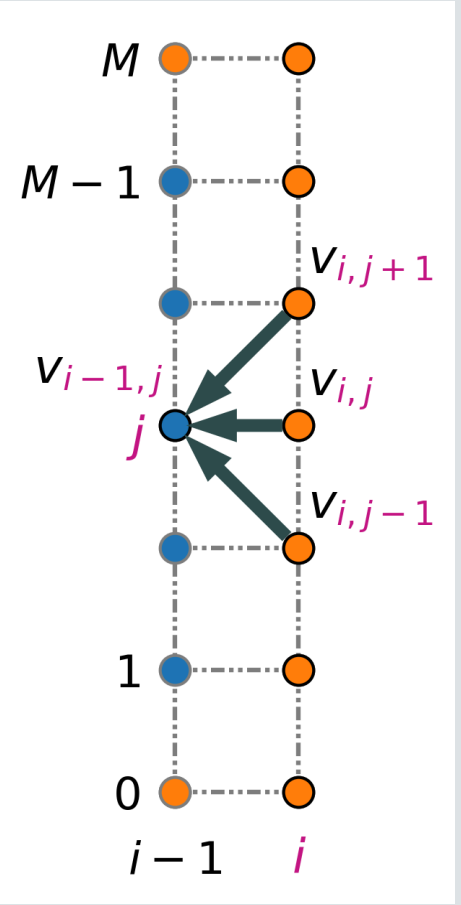
$$\mathbf{v}_{i,1:M-1} - \mathbf{v}_{i-1,1:M-1} + \mathbf{P}\mathbf{v}_{i,0:M} = 0$$

now becomes, with the new notation

$$\mathbf{v}_i^{\Omega} - \mathbf{v}_{i-1}^{\Omega} + \mathbf{A}\mathbf{v}_i^{\Omega} + \mathbf{B}\mathbf{v}_i^{\partial\Omega} = 0$$

The solution $\mathbf{v}_0^{\Omega}$ is obtained by computing

$$\mathbf{v}_{i-1}^{\Omega} = (\mathbf{I} + \mathbf{A})\mathbf{v}_i^{\Omega} + \mathbf{B}\mathbf{v}_i^{\partial\Omega}$$



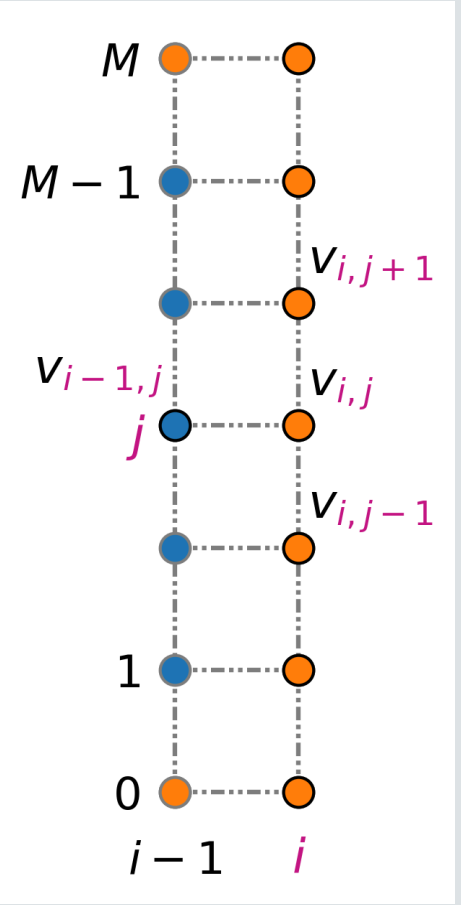
Implicit scheme

If we instead approximate the partial derivative V_t with a **forward difference**

$$V_t(t_i, S_j) \approx \frac{v_{i+1,j} - v_{i,j}}{\Delta_t}$$

then the numerical scheme becomes

$$\mathbf{v}_{i+1}^{\Omega} - \mathbf{v}_i^{\Omega} + \mathbf{A}\mathbf{v}_i^{\Omega} + \mathbf{B}\mathbf{v}_i^{\partial\Omega} = 0$$



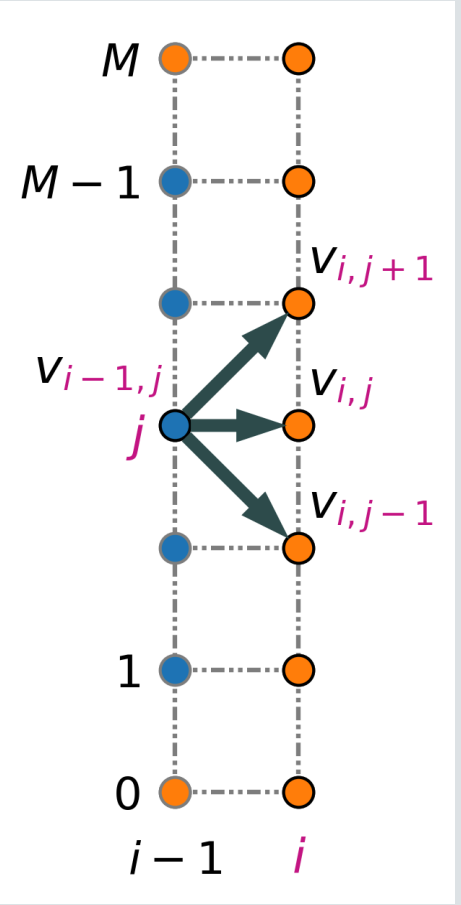
Implicit scheme

$$\mathbf{v}_{i+1}^{\Omega} - \mathbf{v}_{i}^{\Omega} + \mathbf{A}\mathbf{v}_{i}^{\Omega} + \mathbf{B}\mathbf{v}_{i}^{\partial\Omega} = 0$$

and the solution is obtained by solving the linear system

$$(\mathbf{I} - \mathbf{A})\mathbf{v}_{i}^{\Omega} = \mathbf{v}_{i+1}^{\Omega} + \mathbf{B}\mathbf{v}_{i}^{\partial\Omega}$$

for $i = N - 1, \ldots, 1, 0$.
This is an **implicit scheme**.



θ-method

Consider the implicit scheme at time t_i

$$\mathbf{v}_{i+1}^{\Omega} - \mathbf{v}_i^{\Omega} + \mathbf{A}\mathbf{v}_i^{\Omega} + \mathbf{B}\mathbf{v}_i^{\partial\Omega} = 0$$

and the explicit scheme at time t_{i+1}

$$\mathbf{v}_{i+1}^{\Omega} - \mathbf{v}_i^{\Omega} + \mathbf{A}\mathbf{v}_{i+1}^{\Omega} + \mathbf{B}\mathbf{v}_{i+1}^{\partial\Omega} = 0$$

Consider a linear combination of these two equations with $\theta \in [0, 1]$

$$\theta\left(\mathbf{v}_{i+1}^{\Omega} - \mathbf{v}_i^{\Omega} + \mathbf{A}\mathbf{v}_i^{\Omega} + \mathbf{B}\mathbf{v}_i^{\partial\Omega}\right) + (1-\theta)\left(\mathbf{v}_{i+1}^{\Omega} - \mathbf{v}_i^{\Omega} + \mathbf{A}\mathbf{v}_{i+1}^{\Omega} + \mathbf{B}\mathbf{v}_{i+1}^{\partial\Omega}\right) = 0$$

θ-method

After rearranging the terms, we obtain

$$(\mathbf{I} - \theta\mathbf{A})\mathbf{v}_i^{\Omega} = (\mathbf{I} + (1-\theta)\mathbf{A})\mathbf{v}_{i+1}^{\Omega} + \theta\mathbf{B}\mathbf{v}_i^{\partial\Omega} + (1-\theta)\mathbf{B}\mathbf{v}_{i+1}^{\partial\Omega}$$

The solution $\mathbf{v}_0^{\Omega}$ is obtained by solving this linear system for $i = N-1, \ldots, 1, 0$. This numerical PDE scheme is called the **θ-method**.
It generalizes the other schemes:

$\theta = \mathbf{0}$: **explicit scheme**
$\theta = \mathbf{1}$: **implicit scheme**
$\theta = \frac{\mathbf{1}}{\mathbf{2}}$: **Crank-Nicolson scheme**

Implementation of θ-method

Algorithm 1: Compute the solution $\mathbf{v}_0^{\Omega}$ using the θ-method

Inputs : K, r, T, σ, q, $S_{\min}$, $S_{\max}$, N, M, θ

1 Define the terminal conditions $\mathbf{v}_N^{\Omega}$ (final payoff)

2 Define the boundary conditions $\mathbf{v}_i^{\partial\Omega}$, $i = 0, \ldots, N-1$

3 Compute the matrices $\mathbf{A_l} := \mathbf{I} - \theta\mathbf{A}$ and $\mathbf{A_r} := \mathbf{I} + (1-\theta)\mathbf{A}$

4 **for** *i=N-1,...,0* **do**

5 | Solve $\mathbf{A_l}\mathbf{v}_i^{\Omega} = \mathbf{A_r}\mathbf{v}_{i+1}^{\Omega} + \theta B\mathbf{v}_i^{\partial\Omega} + (1-\theta)B\mathbf{v}_{i+1}^{\partial\Omega}$

6 **end**

Outputs: $v_{i,j}$, $i = 0, \ldots, N$, $j = 0, \ldots, M$

Implementation of θ-method in Python (1/2)

```
import numpy as np
from scipy.sparse import diags
from scipy.linalg import solve
from scipy import interpolate

def BS_PUT_theta_method(smin,smax,K,T,r,q,sigma,N,M,theta):
  dt = T/N
  ds = (smax-smin)/M
  tgrid = np.linspace(0.0,T,N+1)
  sgrid = np.linspace(smin,smax,M+1)
  a = dt*sigma*sigma*sgrid*sgrid/(2*ds*ds)
  b = dt*(r-q)*sgrid/(2.0*ds)
  d = a-b
  m = -2.0*a-r*dt
  u = a+b
  A = diags([d[2:M], m[1:M], u[1:(M-1)]], [-1, 0, 1]).toarray()
  B = np.zeros([M-1,2])
  B[0,0] = d[1]
  B[-1,1] = u[M-1]
```

Implementation of θ-method in Python (2/2)

```
Al = np.eye(M-1) - theta*A
Ar = np.eye(M-1) + (1.0-theta)*A

V = np.zeros([N+1,M+1])
V[N,:] = np.maximum(K-sgrid,0.0) # terminal condition
V[:,M] = 0.0 # boundary condition
V[:,0] = K*np.exp(-r*(T-tgrid)) - smin # boundary condition
Vbound = np.zeros([2,N+1])
Vbound[0,:] = V[:,0]
Vbound[1,:] = V[:,M]
for i in np.arange(N-1,-1,-1):
    Br = ((Ar@(V[i+1,1:M])).squeeze()+ \ # line break
         theta*B@(Vbound[:,i])+(1.0-theta)*B@(Vbound[:,i+1])).T
    V[i,1:M] = np.squeeze(solve( Al , Br ))

v0 = interpolate.interp1d(sgrid, V[0,:], kind='cubic')
return v0
```

θ-method examples

```
smin, smax, r, q = 0.0, 150.0, 0.10, 0.00
T, K, sigma = 5/12, 50.0, 0.4
N = 1000 # number of time steps
M = 100 # number of space steps
v0_EX = BS_PUT_theta_method(smin,smax,K,T,r,q,sigma,N,M,0.0) # explicit scheme
v0_CN = BS_PUT_theta_method(smin,smax,K,T,r,q,sigma,N,M,0.5) # Crank-Nicolson sch
v0_IM = BS_PUT_theta_method(smin,smax,K,T,r,q,sigma,N,M,1.0) # implicit scheme

S0 = 50.0
print(BS_PUT(S0,K,T,r-q,sigma)) # price using Black-Scholes formula
## 4.075980984787783
print(v0_EX(S0)) # price using explicit PDE scheme
## 4.075594500192876
print(v0_CN(S0)) # price using Crank-Nicolson scheme
## 4.074951689304087
print(v0_IM(S0)) # price using implicit scheme
## 4.074308638302932
```

θ-method examples

```
smin, smax, r, q = 0.0, 150.0, 0.10, 0.00
T, K, sigma = 5/12, 50.0, 0.4
N = 100 # number of time steps
M = 500 # number of space steps
v0_EX = BS_PUT_theta_method(smin,smax,K,T,r,q,sigma,N,M,0.0) # explicit scheme
v0_CN = BS_PUT_theta_method(smin,smax,K,T,r,q,sigma,N,M,0.5) # Crank-Nicolson sch
v0_IM = BS_PUT_theta_method(smin,smax,K,T,r,q,sigma,N,M,1.0) # implicit scheme

S0 = 50.0
print(BS_PUT(S0,K,T,r-q,sigma)) # price using Black-Scholes formula
## 4.07598098478783
print(v0_EX(S0)) # price using explicit PDE scheme: UNSTABLE
## -2.93647346775972e+154
print(v0_CN(S0)) # price using Crank-Nicolson scheme
## 4.07595711673904
print(v0_IM(S0)) # price using implicit scheme
## 4.06951947435927
```

Numerical stability of PDE schemes

The **explicit scheme** is simple and easy to implement, but is **numerically unstable**. In the case of the Black-Scholes PDE, a **stability analysis** would reveal that it is stable if

$$\frac{\sigma^2 S_{\max}^2 \Delta_t}{\Delta_S^2} \leq 1$$

By contrast, the **implicit scheme** and the **Crank-Nicolson scheme** are **unconditionally stable**. More precisely, the θ-method is unconditionally stable if and only if $\theta \geq 0.5$.

Numerical stability of explicit scheme

```
smin, smax, T, sigma  = 0.0, 150.0, 5/12, 0.4
N = 1000 # number of time steps
M = 100 # number of space steps
dt, ds = T/N, (smax-smin)/M
print(sigma*sigma*smax*smax*dt/(ds*ds))
## 0.666666666666667
```

In the first example, the stability ratio is smaller than 1 $\Rightarrow$ The explicit scheme is **stable**

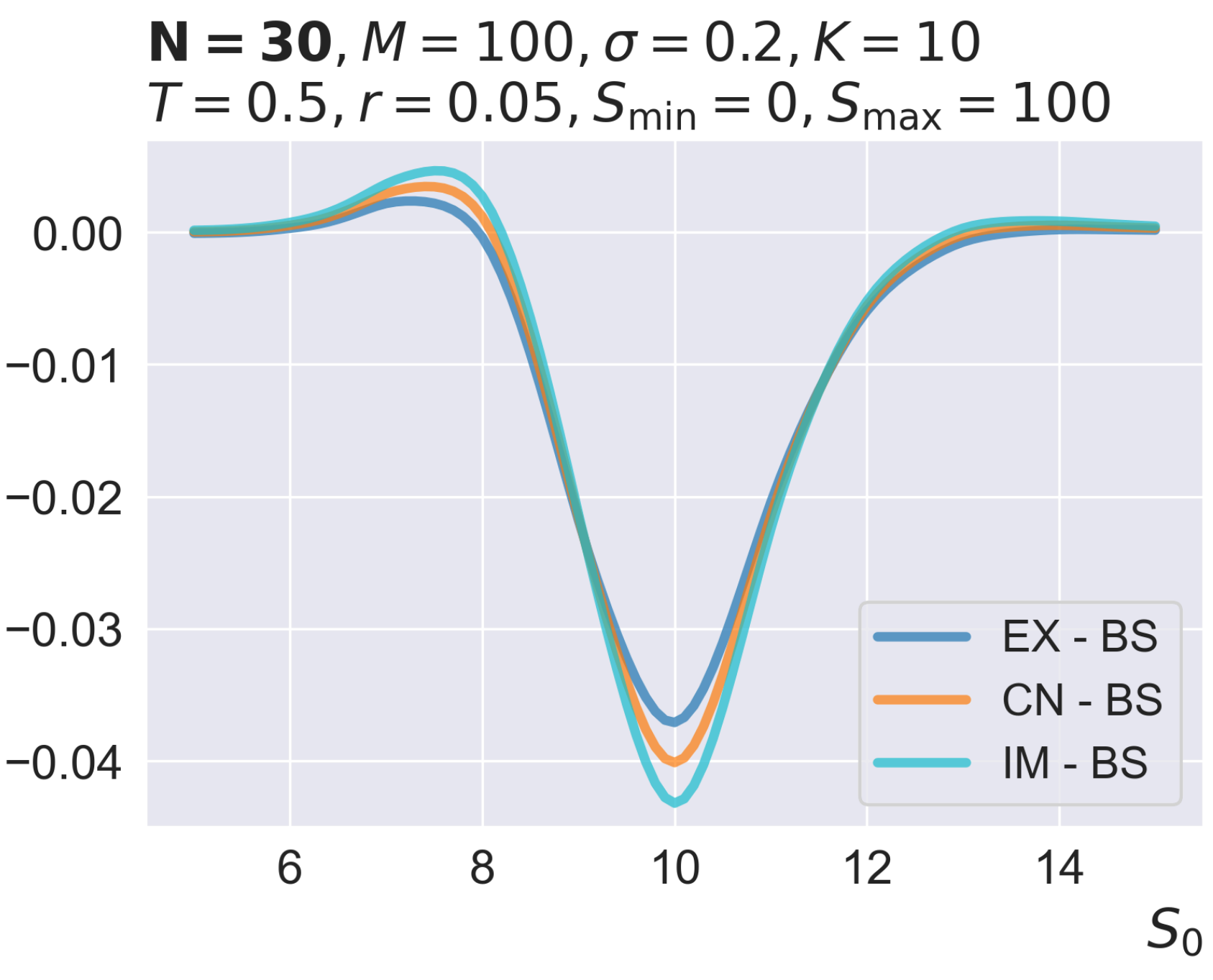
```
smin, smax, T, sigma  = 0.0, 150.0, 5/12, 0.4
N = 100 # number of time steps
M = 500 # number of space steps
dt, ds = T/N, (smax-smin)/M
print(sigma*sigma*smax*smax*dt/(ds*ds))
## 166.66666666666669
```

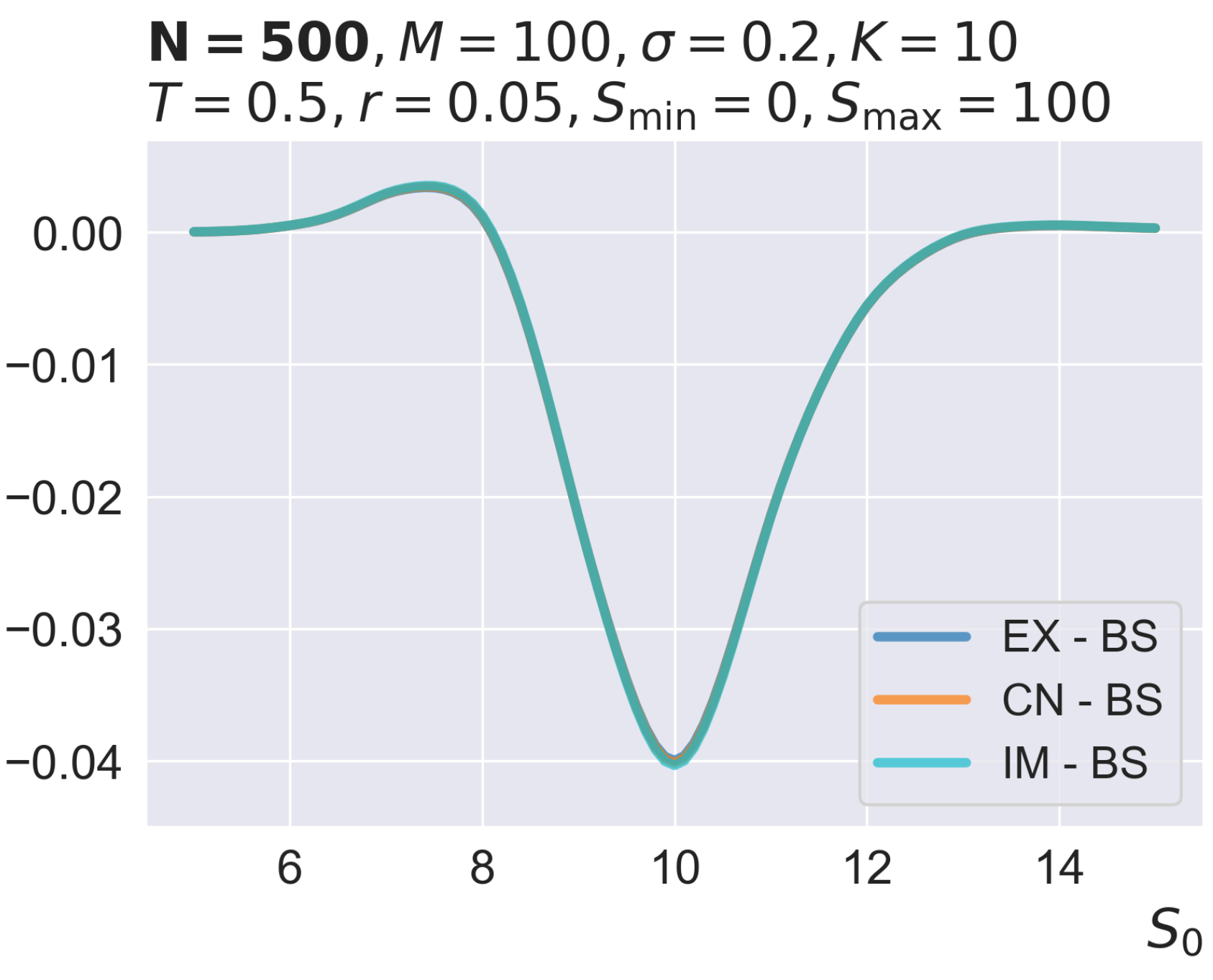
In the second example, the stability ratio is greater than 1 $\Rightarrow$ The explicit scheme is **unstable**

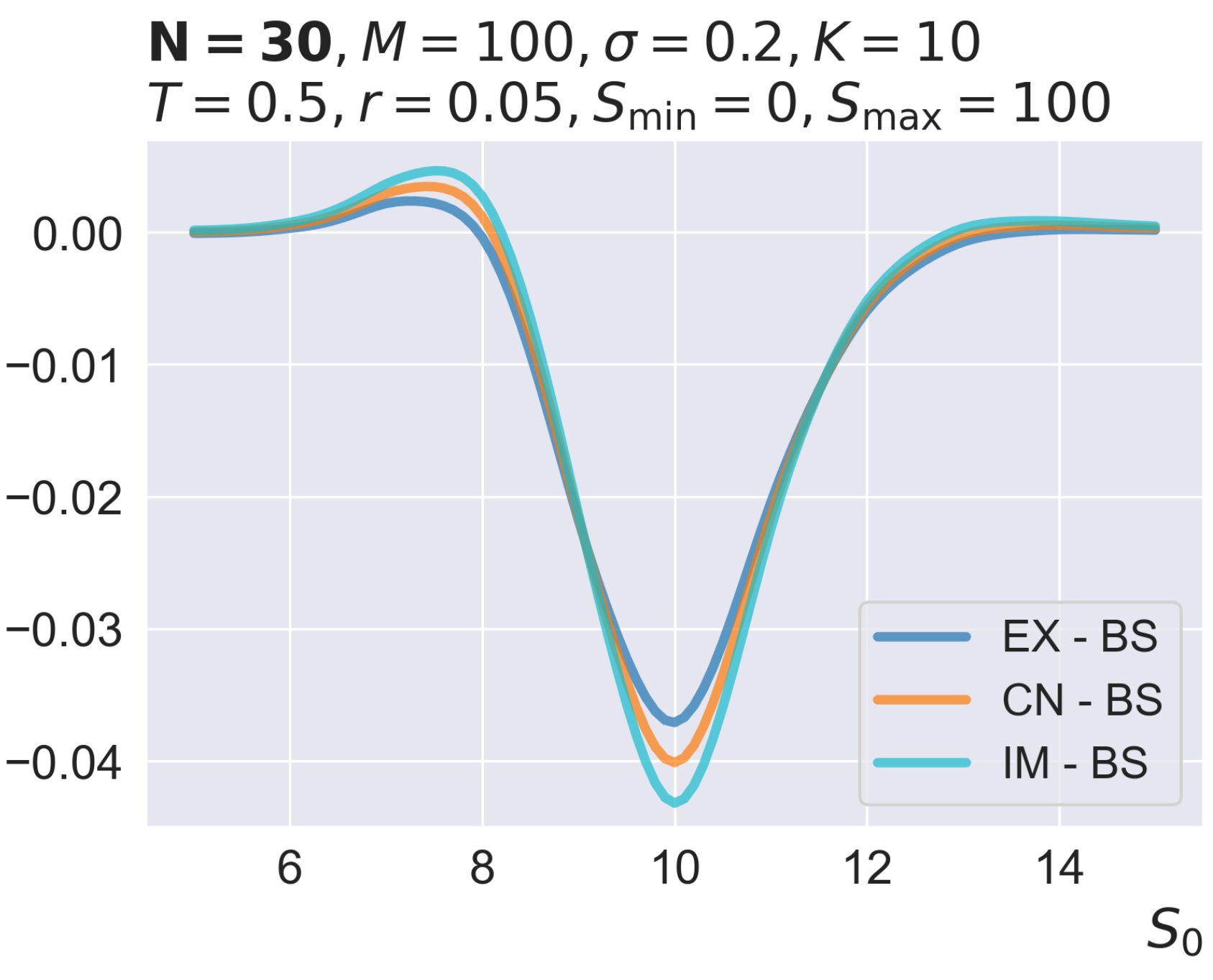
Numerical stability of Crank-Nicolson scheme

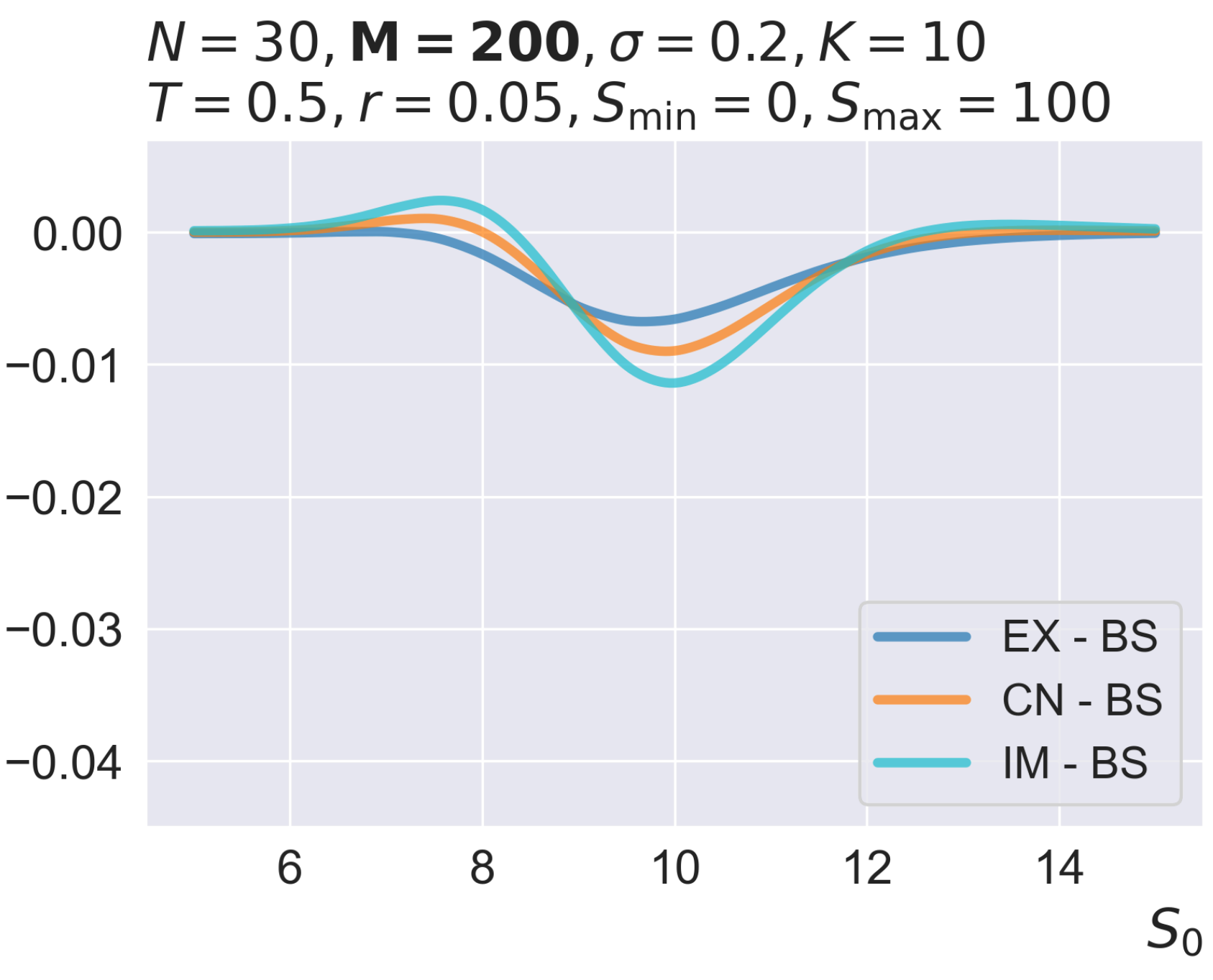
The θ method is **unconditionally stable** for any $\theta \geq 0.5$, therefore the Crank-Nicolson scheme $\theta = 0.5$ is stable. However, the θ method is only **conditionally stable** for $\theta = 0.5 - \varepsilon$ for all $0 < \varepsilon \leq 0.5$. This means that any perturbation, for example due to **float-rounding error**, could push the Crank-Nicolson scheme in the conditionally stable region, and even generate unstable oscillations if the stability condition is not satisfied.

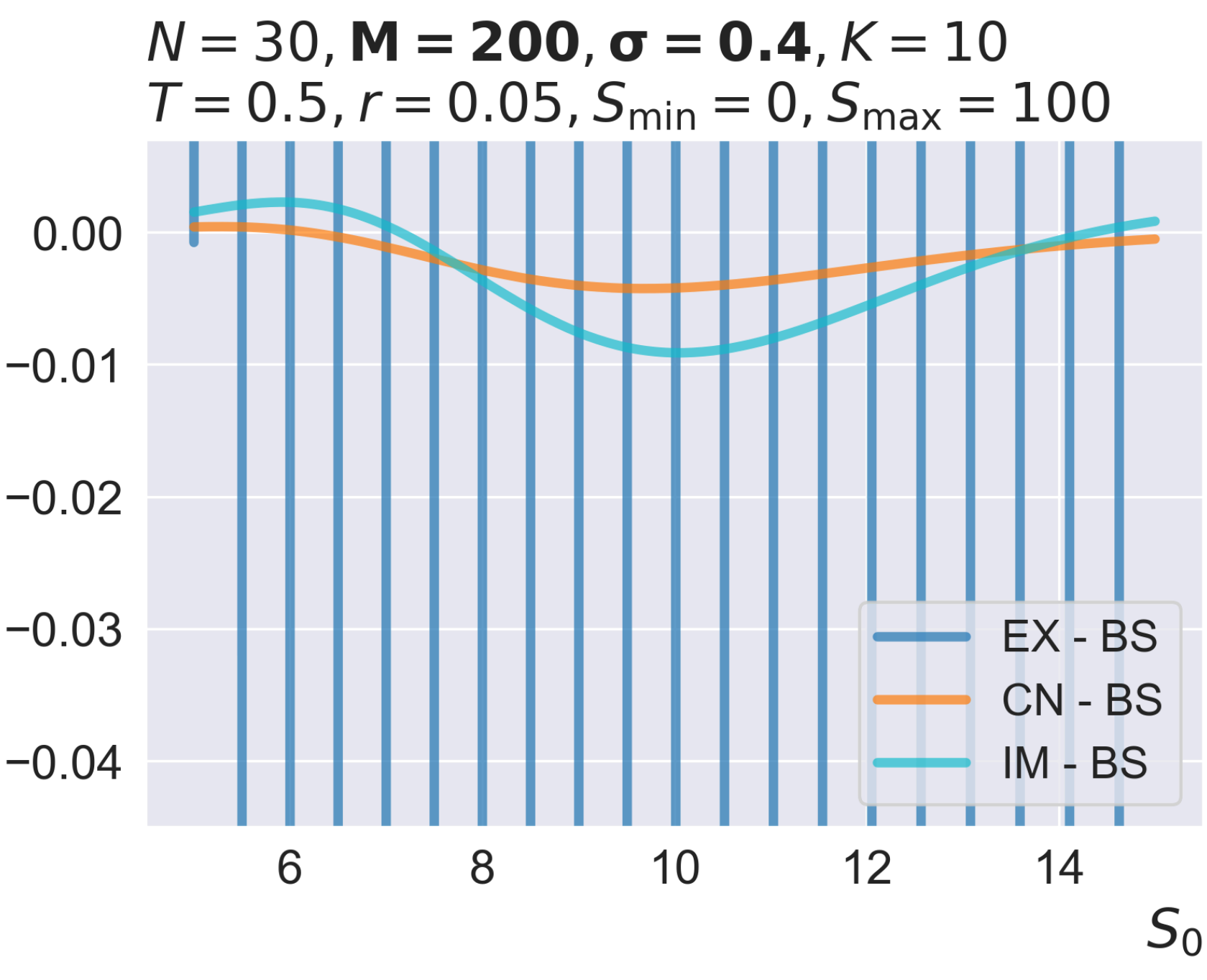
For this reason, the Crank-Nicolson scheme is called **neutrally stable**. To produce the following numerical results, the θ of the "Crank-Nicolson" scheme was set to $0.5 + \varepsilon$ for a small $\varepsilon > 0$ whenever small oscillations were visible in the results. When such unstable oscillations occur, they are obviously negligible compared to the ones of the explicit scheme.

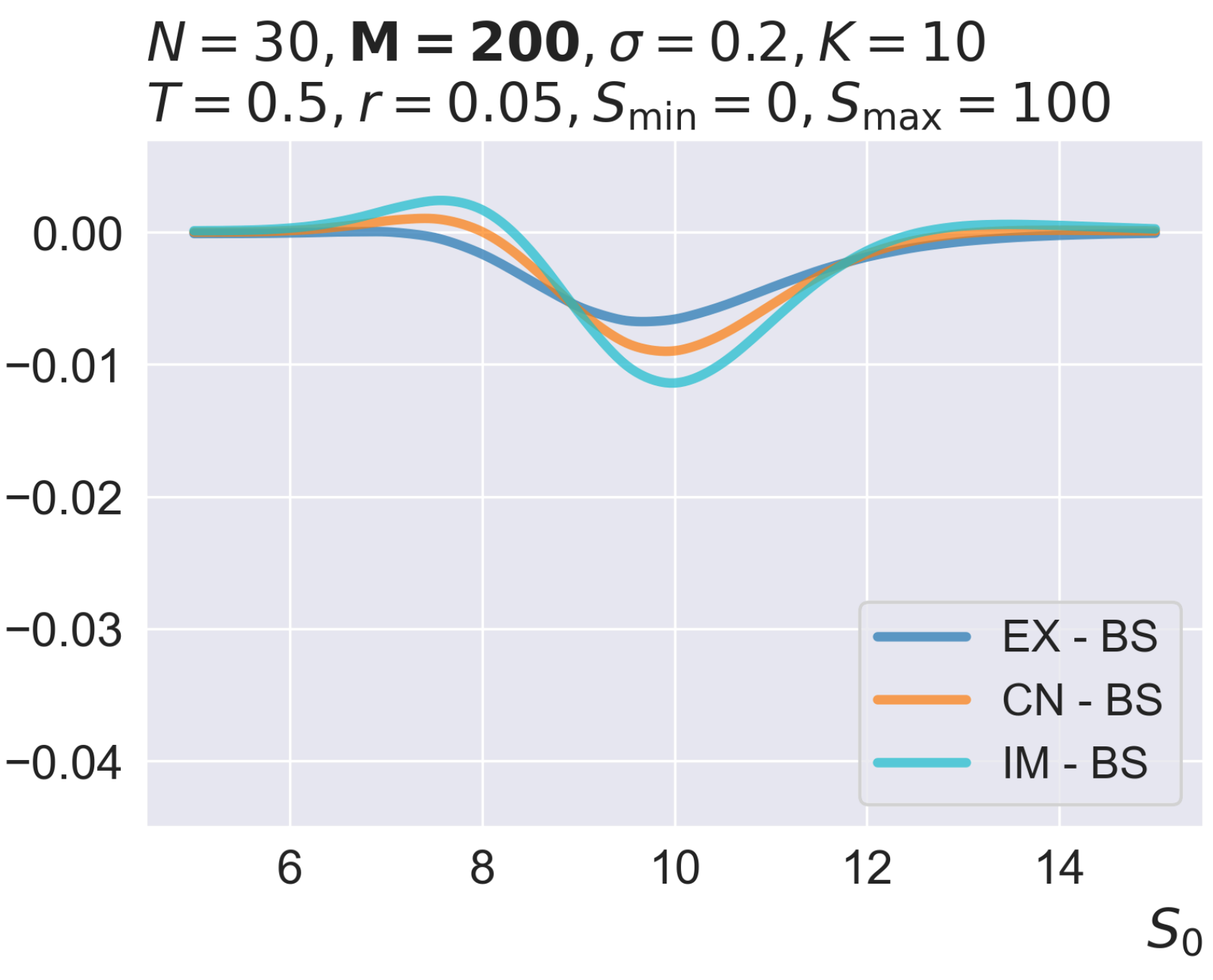

N = 30, M = 100, σ = 0.2, K = 10
T = 0.5, r = 0.05, S_min = 0, S_max = 100
0.00
−0.01
−0.02
−0.03
−0.04
6
8
10
12
14
S_0
EX - BS
CN - BS
IM - BS

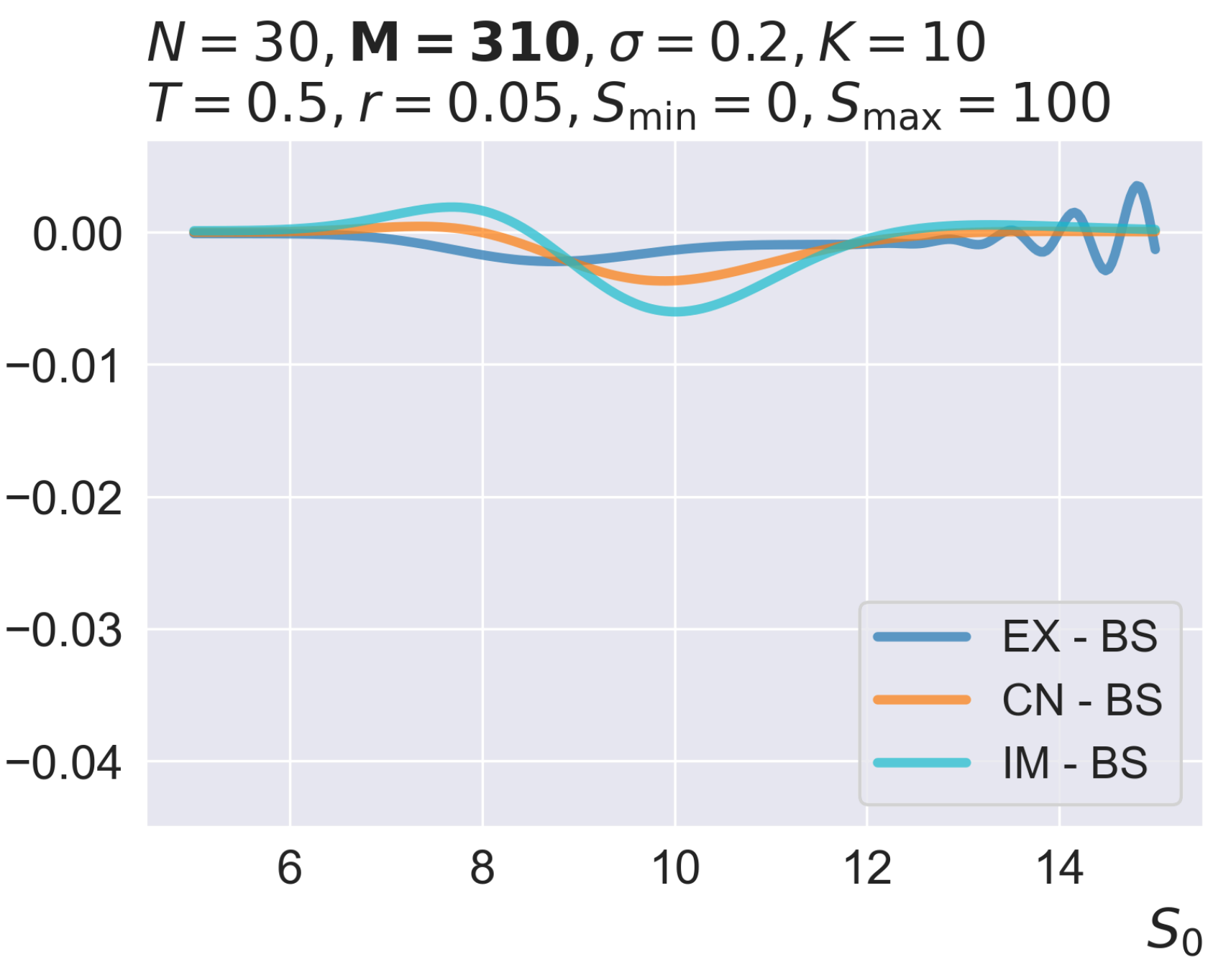

N = 500, M = 100, σ = 0.2, K = 10
T = 0.5, r = 0.05, S_min = 0, S_max = 100
0.00
−0.01
−0.02
−0.03
−0.04
6
8
10
12
14
S_0
EX - BS
CN - BS
IM - BS

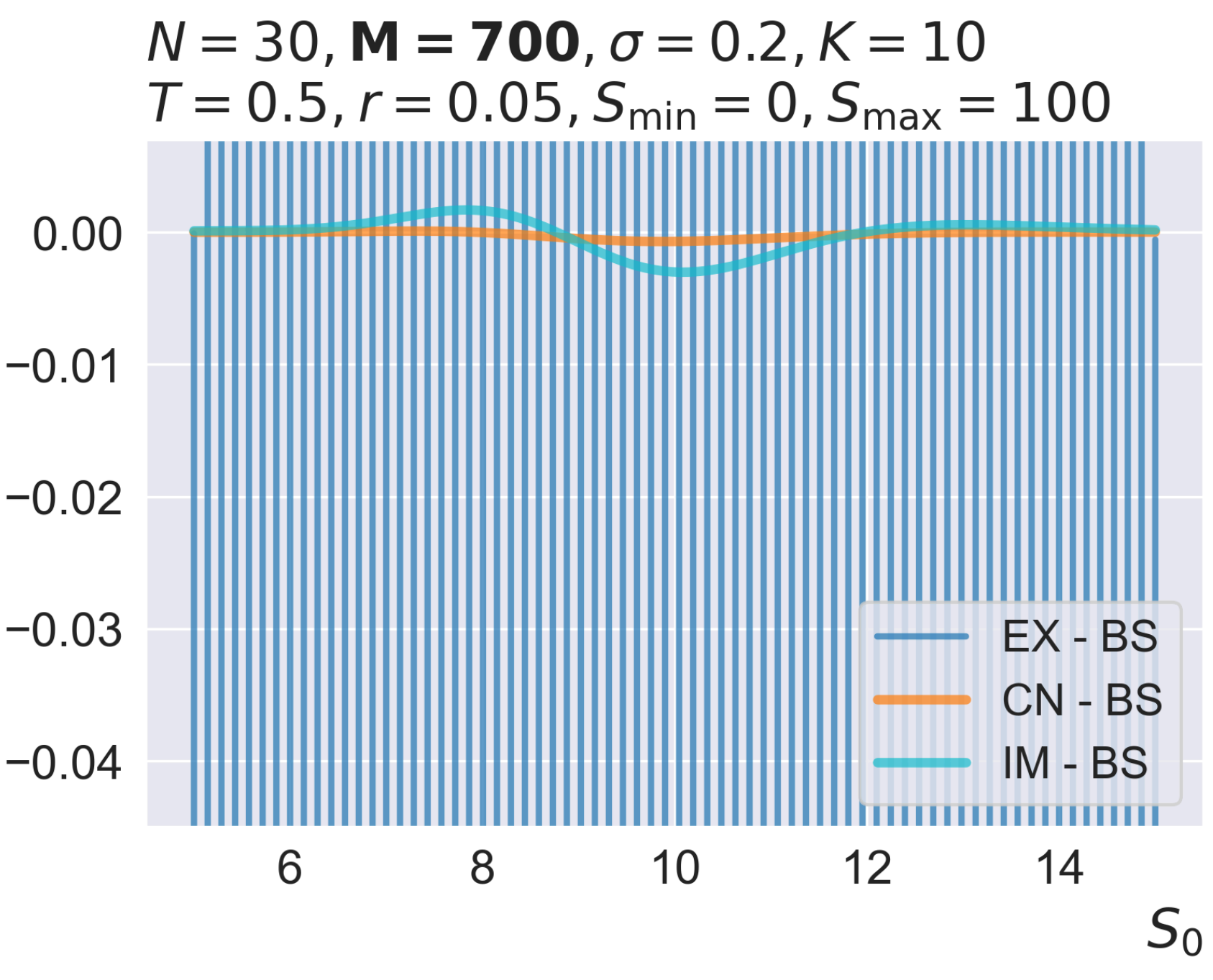

N = 30, M = 100, σ = 0.2, K = 10
T = 0.5, r = 0.05, S_min = 0, S_max = 100
0.00
−0.01
−0.02
−0.03
−0.04
6
8
10
12
14
S_0
EX - BS
CN - BS
IM - BS

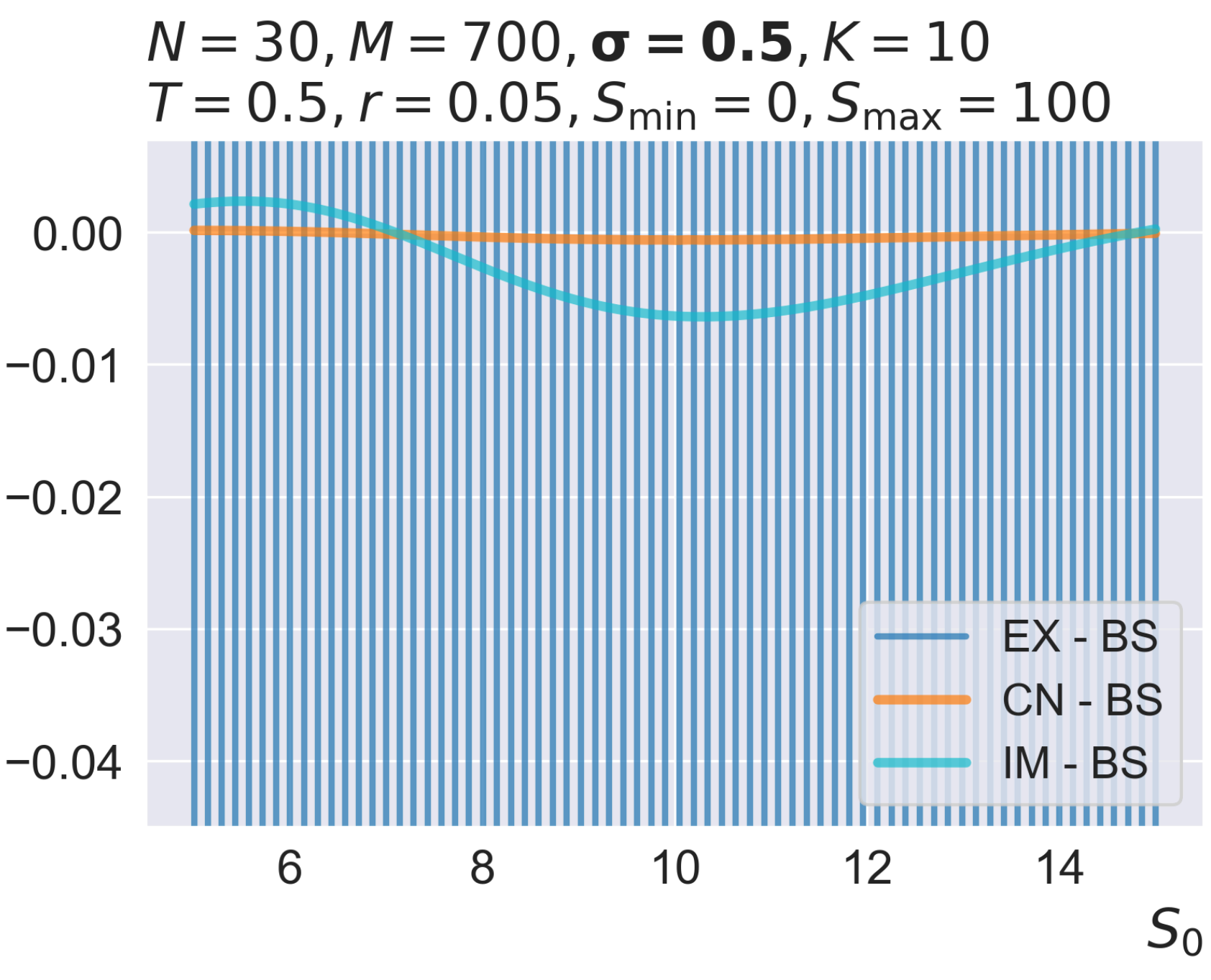

$N = 30, \mathbf{M = 200}, \sigma = 0.2, K = 10$
$T = 0.5, r = 0.05, S_{\min} = 0, S_{\max} = 100$
0.00
−0.01
−0.02
−0.03
−0.04
6
8
10
12
14
S_0
EX - BS
CN - BS
IM - BS


$N = 30, \mathbf{M = 200}, \boldsymbol{\sigma = 0.4}, K = 10$
$T = 0.5, r = 0.05, S_{\min} = 0, S_{\max} = 100$
0.00
−0.01
−0.02
−0.03
−0.04
6
8
10
12
14
S_0
EX - BS
CN - BS
IM - BS


$N = 30, \mathbf{M = 200}, \sigma = 0.2, K = 10$
$T = 0.5, r = 0.05, S_{\min} = 0, S_{\max} = 100$
0.00
−0.01
−0.02
−0.03
−0.04
6
8
10
12
14
S_0
EX - BS
CN - BS
IM - BS


$N = 30, \mathbf{M = 310}, \sigma = 0.2, K = 10$
$T = 0.5, r = 0.05, S_{\min} = 0, S_{\max} = 100$
0.00
−0.01
−0.02
−0.03
−0.04
6
8
10
12
14
S_0
EX - BS
CN - BS
IM - BS


$N = 30, \mathbf{M = 700}, \sigma = 0.2, K = 10$
$T = 0.5, r = 0.05, S_{\min} = 0, S_{\max} = 100$
0.00
−0.01
−0.02
−0.03
−0.04
6
8
10
12
14
S_0
EX - BS
CN - BS
IM - BS


$N = 30, M = 700, \boldsymbol{\sigma = 0.5}, K = 10$
$T = 0.5, r = 0.05, S_{\min} = 0, S_{\max} = 100$
0.00
−0.01
−0.02
−0.03
−0.04
6
8
10
12
14
S_0
EX - BS
CN - BS
IM - BS

Observations

▷ The explicit scheme is unreliable (*numerical instability*).

▷ The **Crank-Nicolson** scheme is the most accurate, and is the only one which improves when M grows for fixed N.

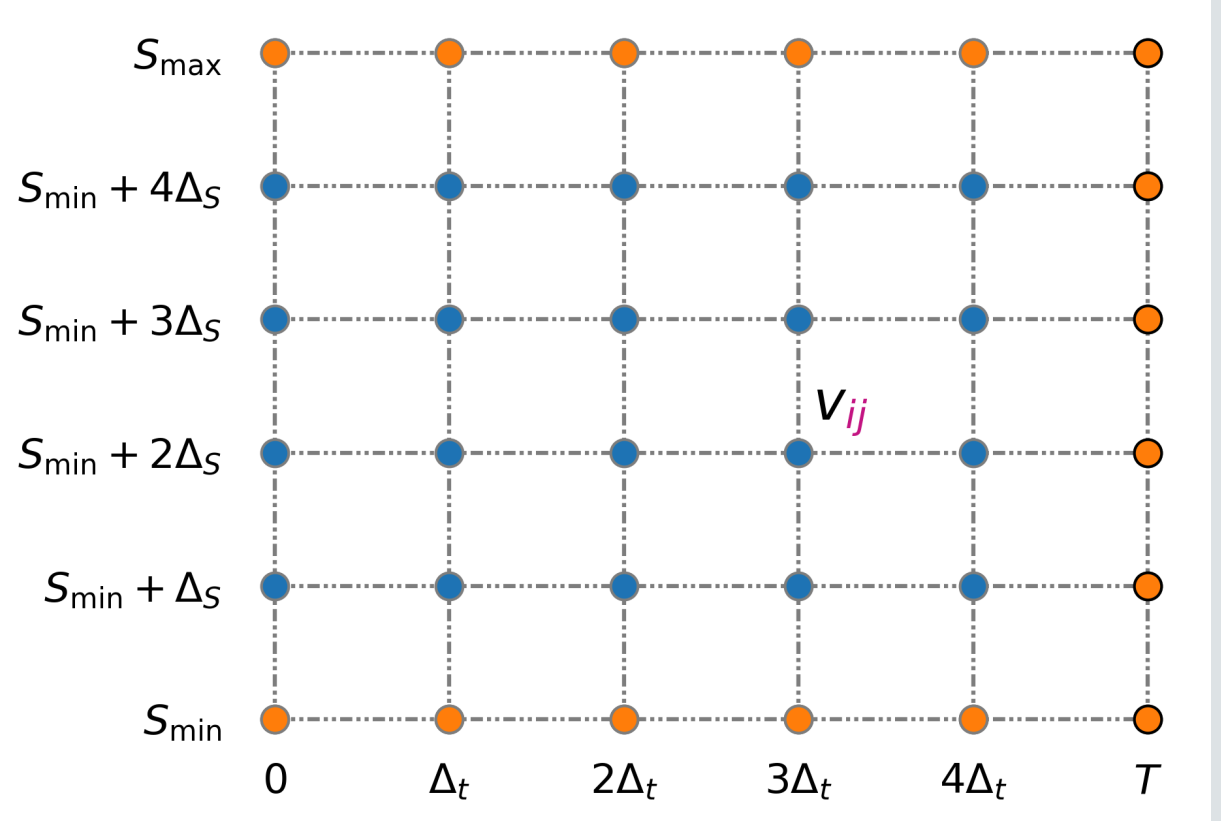
▷ Errors grow with σ.

▷ Errors are larger *at-the-money* (when the price S is close to the strike price K). This suggests to use a variable space grid (finer grid around K).

3.4 The Black-Scholes PDE with change of variable

Coordinate transform of space variable

One can perform a change of variable in the space dimension before attempting to solve the PDE.



Coordinate transform of space variable

Recall the Black-Scholes PDE

$$V_t + \frac{1}{2}\sigma^2 S^2 V_{SS} + (r-q)SV_S - rV = 0$$

Consider the transform $S = S(x)$ and plug it in the PDE.

We choose the transform $S(x)$ such that it is analytical, twice-differentiable and such that the derivative functions $S_x(x)$ and $S_{xx}(x)$ are known analytically.

Coordinate transform of space variable

The partial derivatives V_S and V_{SS} become (use the **chain rule**)

$$V_x = \frac{\partial V}{\partial S}\frac{\partial S}{\partial x} = V_S S_x \quad \text{therefore} \quad \mathbf{V_S = V_x / S_x}$$

$$\frac{\partial V_S}{\partial x} = \frac{\partial V_S}{\partial S}\frac{\partial S}{\partial x} = V_{SS} S_x \quad \text{therefore}$$

$$\mathbf{V_{SS} = \frac{1}{S_x}\frac{\partial}{\partial x}\left(\frac{V_x}{S_x}\right) = \frac{V_{xx}S_x - V_x S_{xx}}{S_x^3}}$$

Black-Scholes PDE after change of variable

In terms of the variables t and x the Black-Scholes PDE becomes

$$V_t + \frac{1}{2}\sigma^2 \left(\frac{S}{S_x}\right)^2 \frac{V_{xx}S_x - V_x S_{xx}}{S_x} + (r-q)\frac{S}{S_x}V_x - rV = 0$$

where S, S_x and S_{xx} are analytical functions of x.
Define $\Delta_x = (x_{\max} - x_{\min})/M$ and $\Delta_t = T/N$
The numerical approximation v_{ij} of $V(t,x)$ is computed at the grid points

$$(t_i, x_j) = (i\Delta_t, x_{\min} + j\Delta_x)$$

$$v_{ij} \approx V(i\Delta_t, x_{\min} + j\Delta_x)$$

Finite-difference after change of variable

We approximate the partial derivative V_t by forward difference, and the partial derivatives V_x and V_{xx} by centered difference

$$V_t(t_i, x_j) \approx \frac{v_{i+1,j} - v_{i,j}}{\Delta_t}$$

$$V_x(t_i, x_j) \approx \frac{v_{i,j+1} - v_{i,j-1}}{2\Delta_x}$$

$$V_{xx}(t_i, x_j) \approx \frac{v_{i,j+1} - 2v_{ij} + v_{i,j-1}}{\Delta_x^2}$$

Finite-difference after change of variable

The finite-difference scheme for the transformed Black-Scholes PDE is

$$\frac{v_{i+1,j}-v_{i,j}}{\Delta_t} + \frac{1}{2}\sigma^2\left(\frac{S(x_j)}{S_x(x_j)}\right)^2\left(\frac{v_{i,j+1}-2v_{ij}+v_{i,j-1}}{\Delta_x^2} - \frac{v_{i,j+1}-v_{i,j-1}}{2\Delta_x}\frac{S_{xx}(x_j)}{S_x(x_j)}\right)$$
$$+(r-q)\frac{S(x_j)}{S_x(x_j)}\frac{v_{i,j+1}-v_{i,j-1}}{2\Delta_x} - rv_{ij} = 0$$

Recall that the change of variable $S(x)$ is chosen such that the derivatives $S_x(x)$ and $S_{xx}(x)$ are explicit analytical functions.

Finite-difference after change of variable

Define $a_j = \frac{\Delta_t \sigma^2 S^2(x_j)}{2\Delta_x^2 S_x^2(x_j)}$ and $b_j = \frac{\Delta_t S(x_j)}{2\Delta_x S_x(x_j)}\left[r - q - \frac{\sigma^2 S(x_j) S_{xx}(x_j)}{2S_x^2(x_j)}\right]$. We get

$$v_{i+1,j} - v_{i,j} + a_j(v_{i,j+1} - 2v_{ij} + v_{i,j-1}) + b_j(v_{i,j+1} - v_{i,j-1}) - r\Delta_t v_{ij} = 0$$

for $j = 1, \ldots, M-1$. Grouping similar terms together we get

$$v_{i+1,j} - v_{i,j} + d_j v_{i,j-1} + m_j v_{ij} + u_j v_{i,j+1} = 0$$

where $d_j := a_j - b_j$, $m_j := -2a_j - r\Delta_t$ and $u_j := a_j + b_j$

Implicit scheme after change of variables

Using the same matrix notation as before, this implicit scheme reads

$$(\mathbf{I} - \mathbf{A})\mathbf{v}_i^{\Omega} = \mathbf{v}_{i+1}^{\Omega} + \mathbf{B}\mathbf{v}_i^{\partial\Omega}$$

where the matrices $\mathbf{A}$ and $\mathbf{B}$ are once again given by

$$\mathbf{A} = \begin{bmatrix} m_1 & u_1 & 0 & \cdots & \cdots \\ d_2 & m_2 & u_2 & & \\ & d_3 & \ddots & \ddots & \\ & & \ddots & \ddots & u_{M-2} \\ \cdots & \cdots & 0 & d_{M-1} & m_{M-1} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} d_1 & 0 \\ 0 & \vdots \\ \vdots & \vdots \\ \vdots & 0 \\ 0 & u_{M-1} \end{bmatrix}$$

Choice of transform $S = S(x)$

One possible choice is the **log-transform**

$$S(x) = Ke^x$$

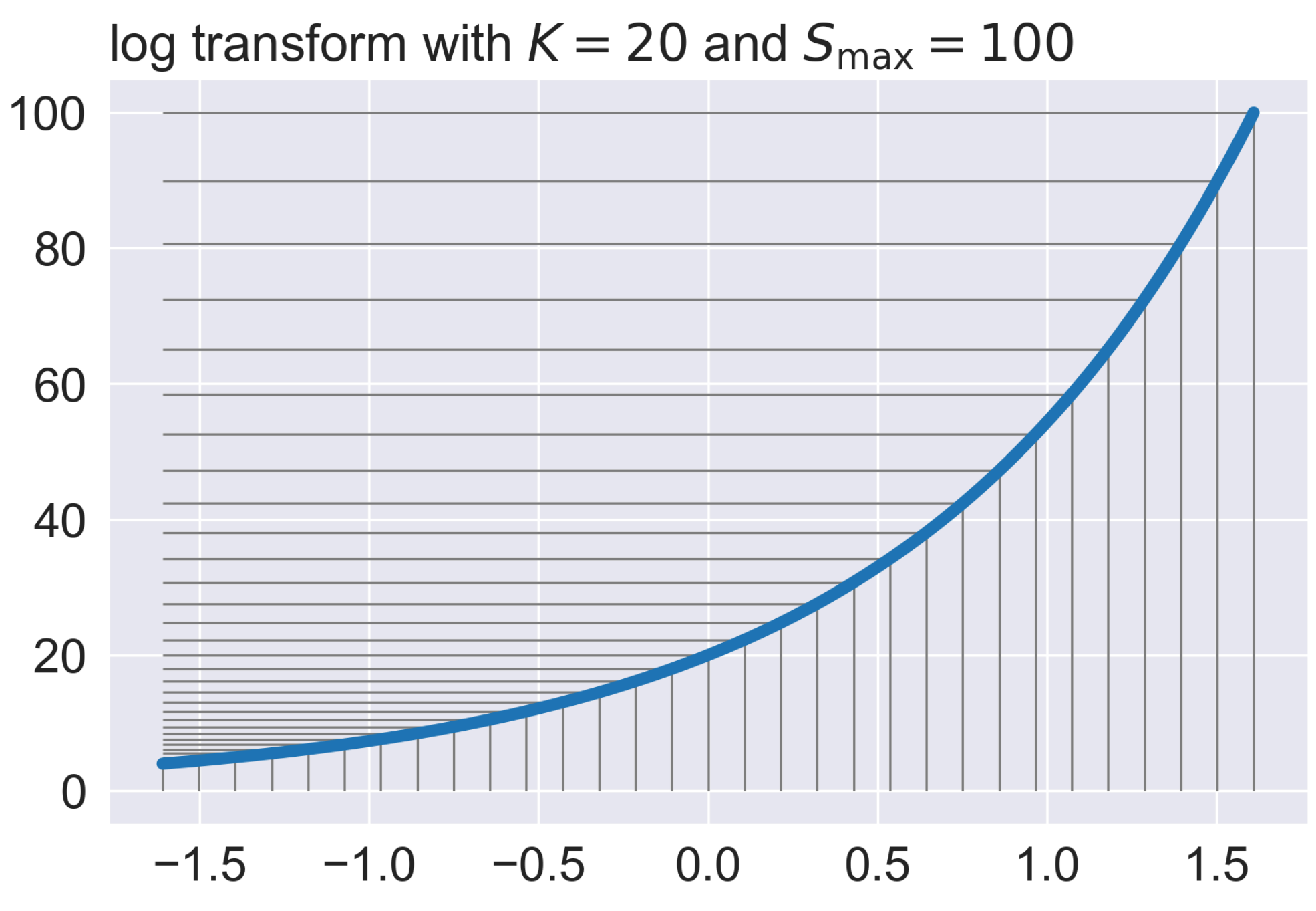
with $x \in [x_{\min}, x_{\max}]$. These bounds are chosen such that

$$S_{\max} = Ke^{x_{\max}}, \qquad x_{\min} = -x_{\max}$$

With this choice of transform,

$$S(x) = S_x(x) = S_{xx}(x) = Ke^x$$

so a_j and b_j simplify into $a_j = \frac{\Delta_t \sigma^2}{2\Delta_x^2}$ and $b_j = \frac{\Delta_t}{2\Delta_x}\left[r - q - \frac{\sigma^2}{2}\right]$.


log transform with $K = 20$ and $S_{max} = 100$
100
80
60
40
20
0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5

Choice of transform $S = S(x)$

Another possible choice is the **sinh-transform**

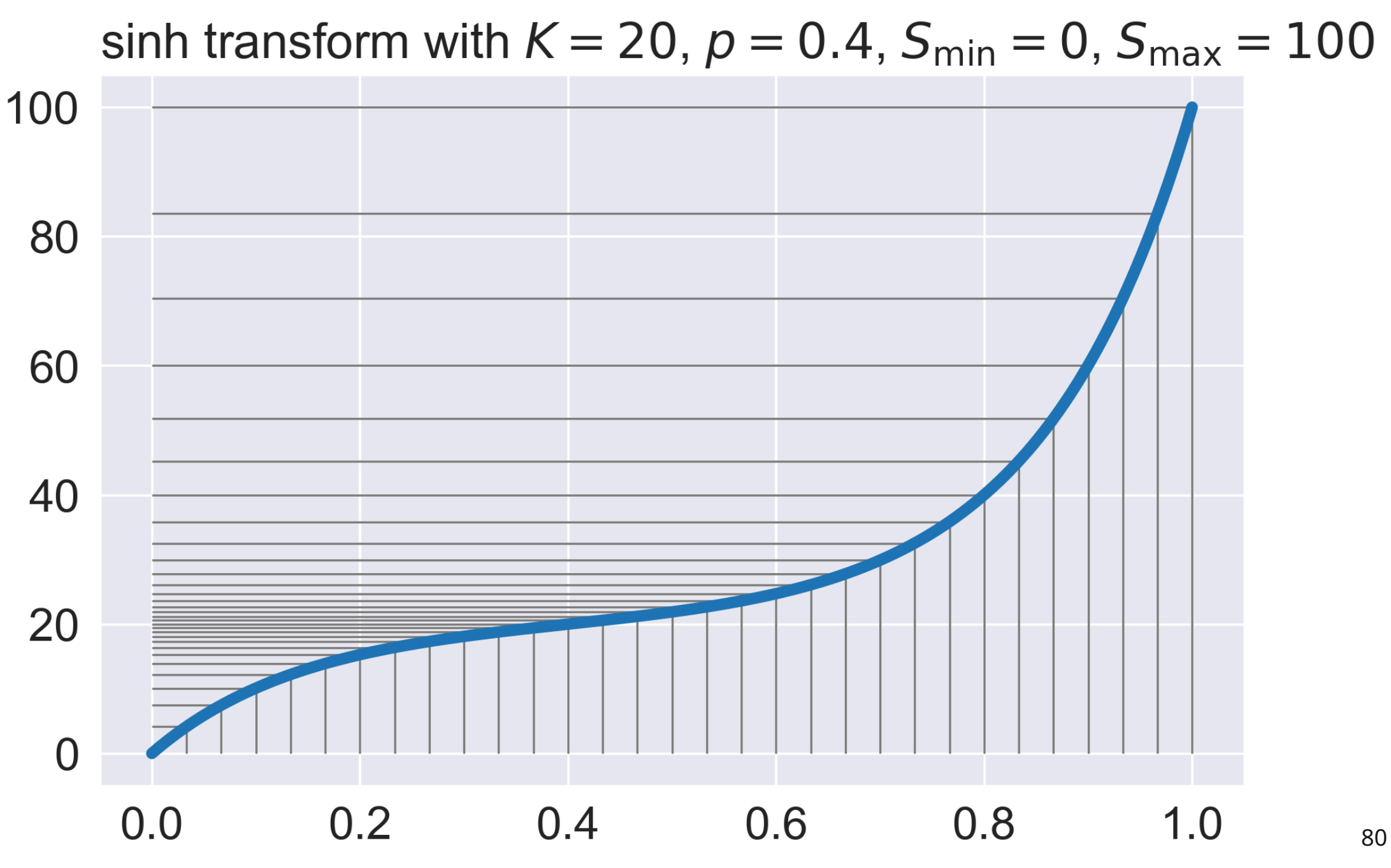
$$S(x) = K + \frac{K}{2\lambda}\sinh\left(\frac{cx}{p} - c\right)$$

for $x \in [0, 1]$, where $\sinh$ is the hyperbolic sine

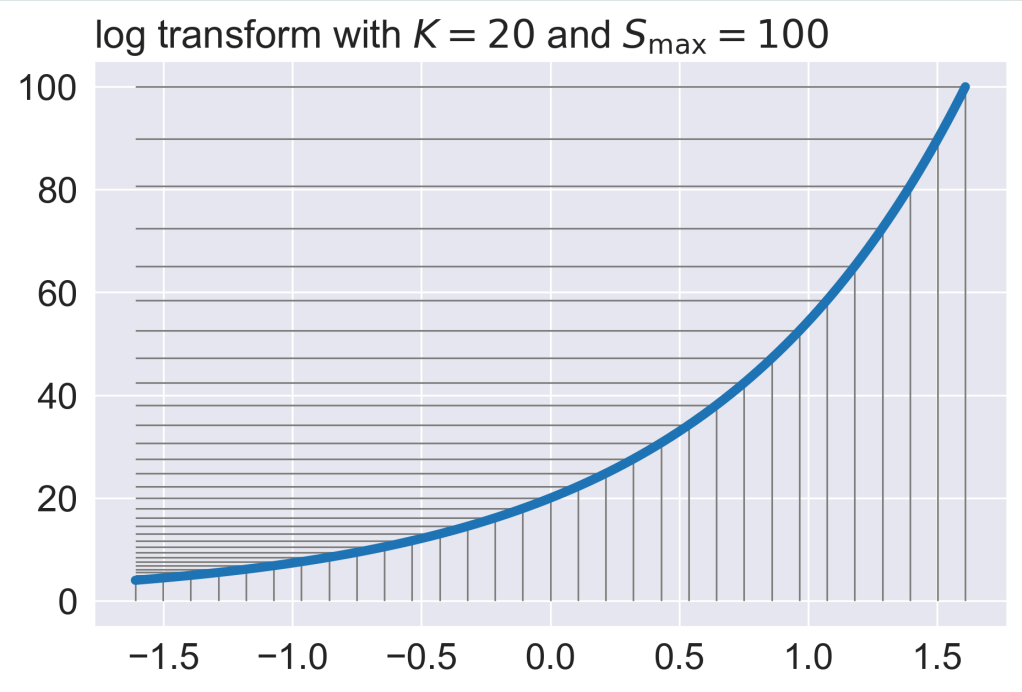
$$\sinh(x) = (e^x - e^{-x})/2$$

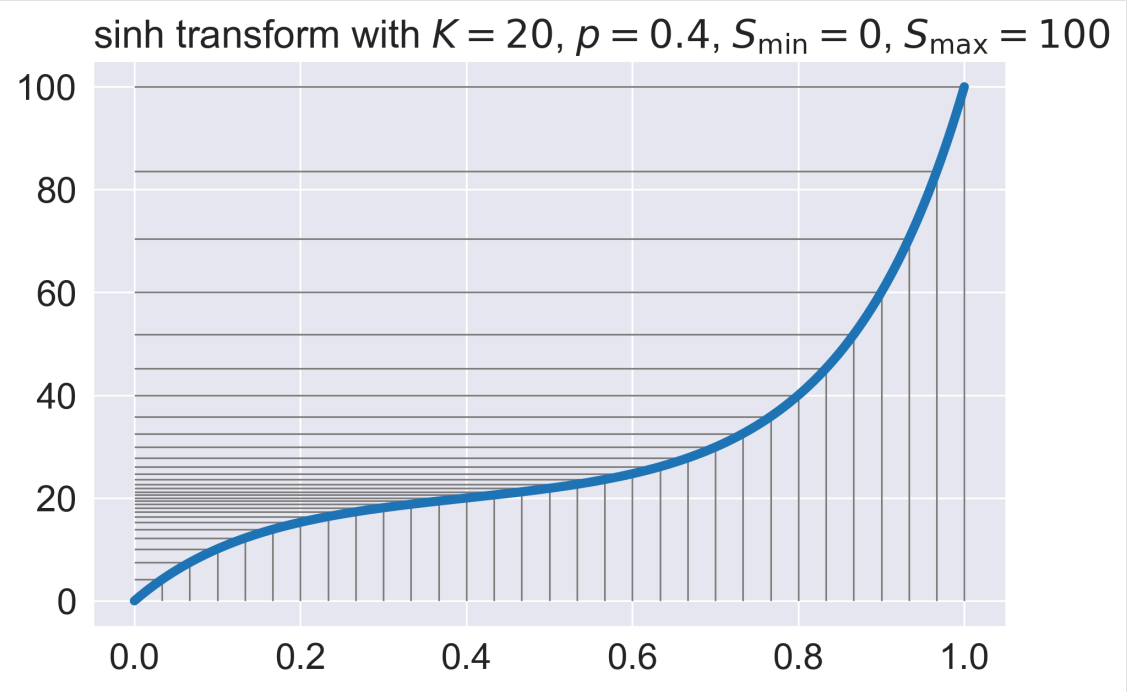
p is a shape parameter such that $S(p) = K$.
λ and c are shape parameters chosen such that

$$S(0) = S_{\min} \quad \text{and} \quad S(1) = S_{\max}$$


sinh transform with $K = 20$, $p = 0.4$, $S_{\min} = 0$, $S_{\max} = 100$
100
80
60
40
20
0
0.0
0.2
0.4
0.6
0.8
1.0

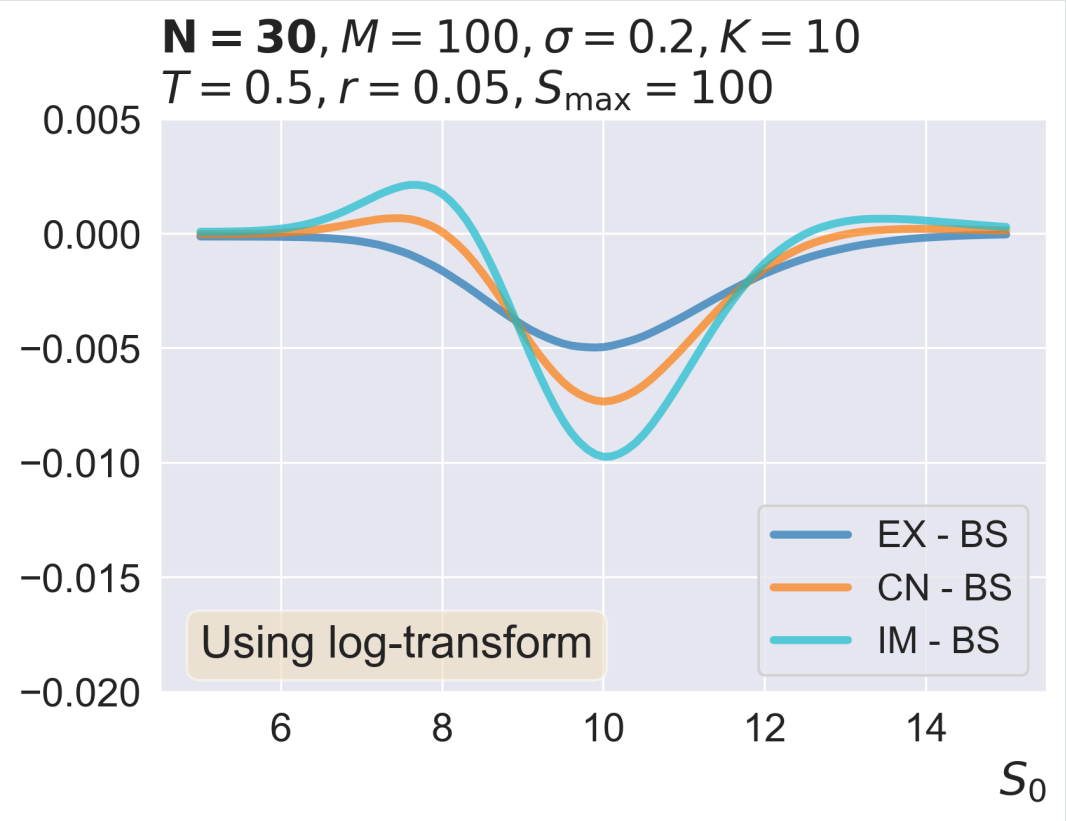
log-transform vs. sinh-transform

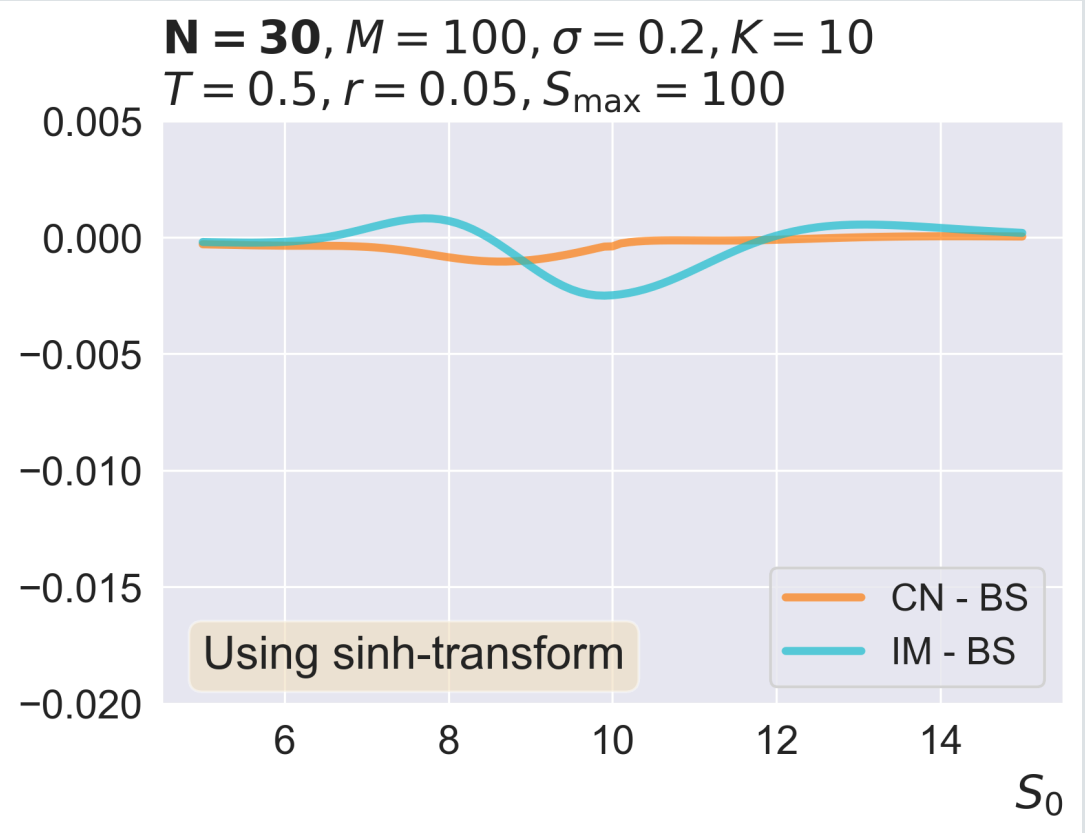




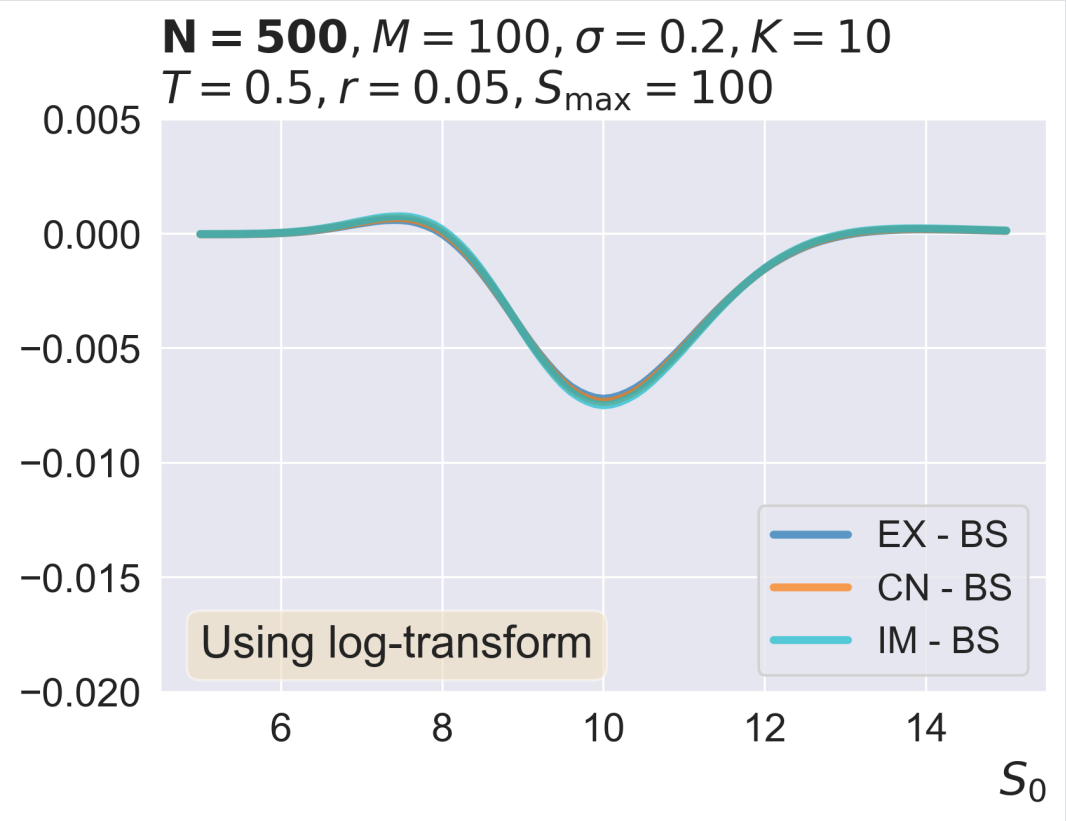
The smaller the prices S the finer the log grid. Instead, the sinh grid is finer around the strike price K.

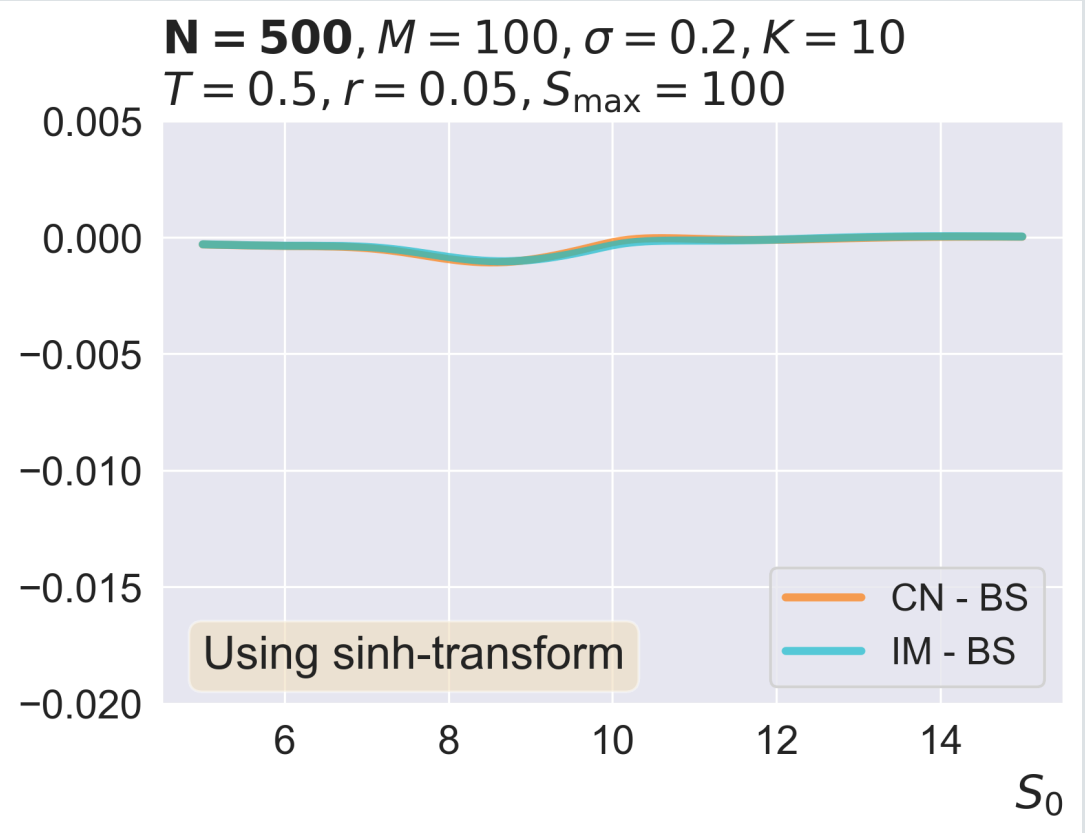
log-transform vs. sinh-transform



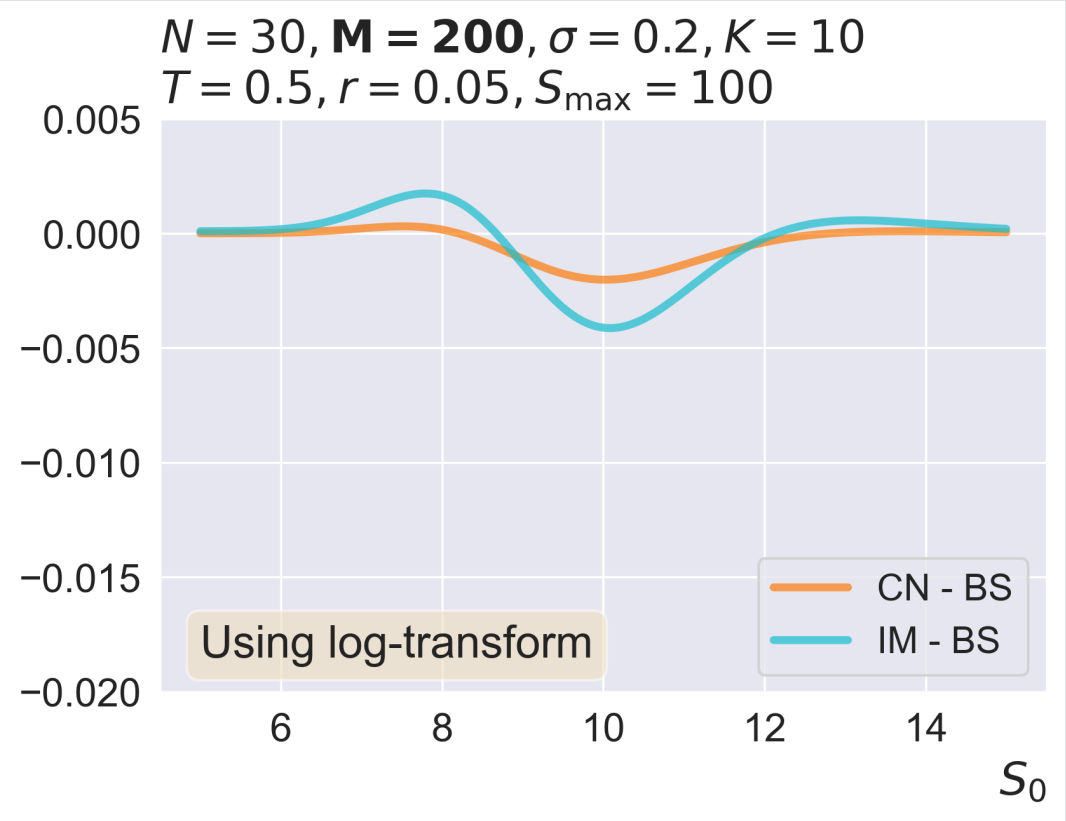


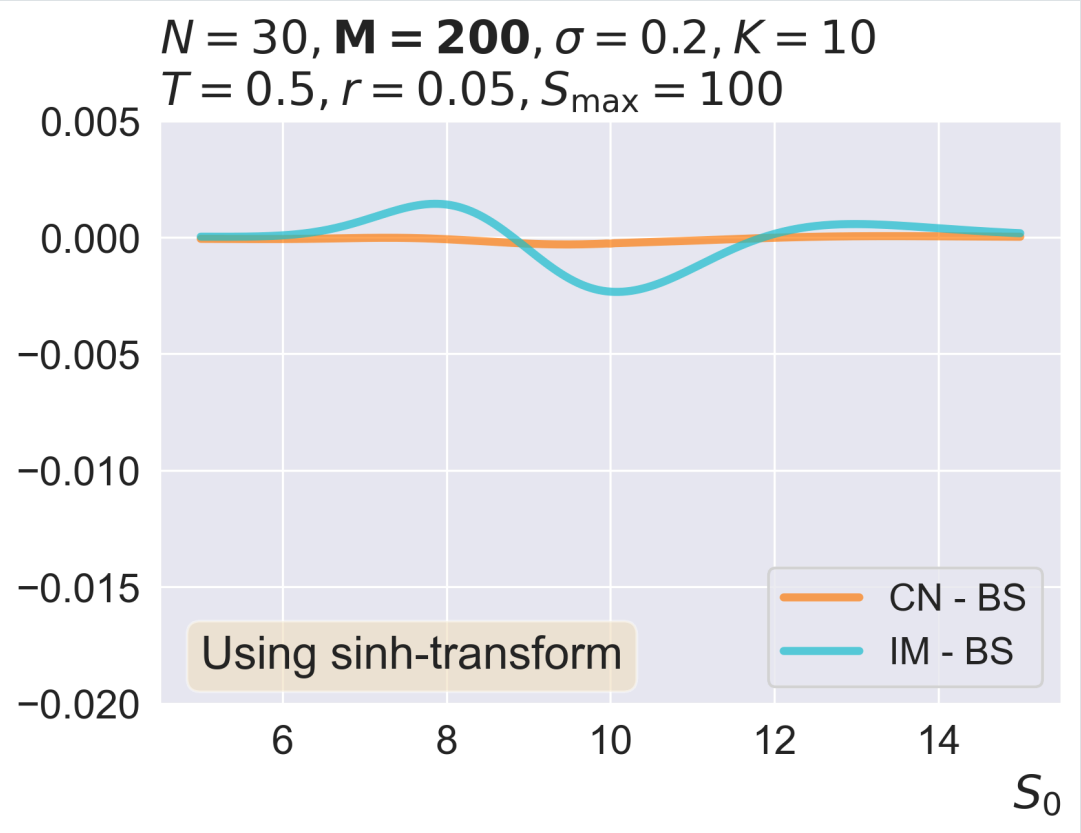
log-transform vs. sinh-transform



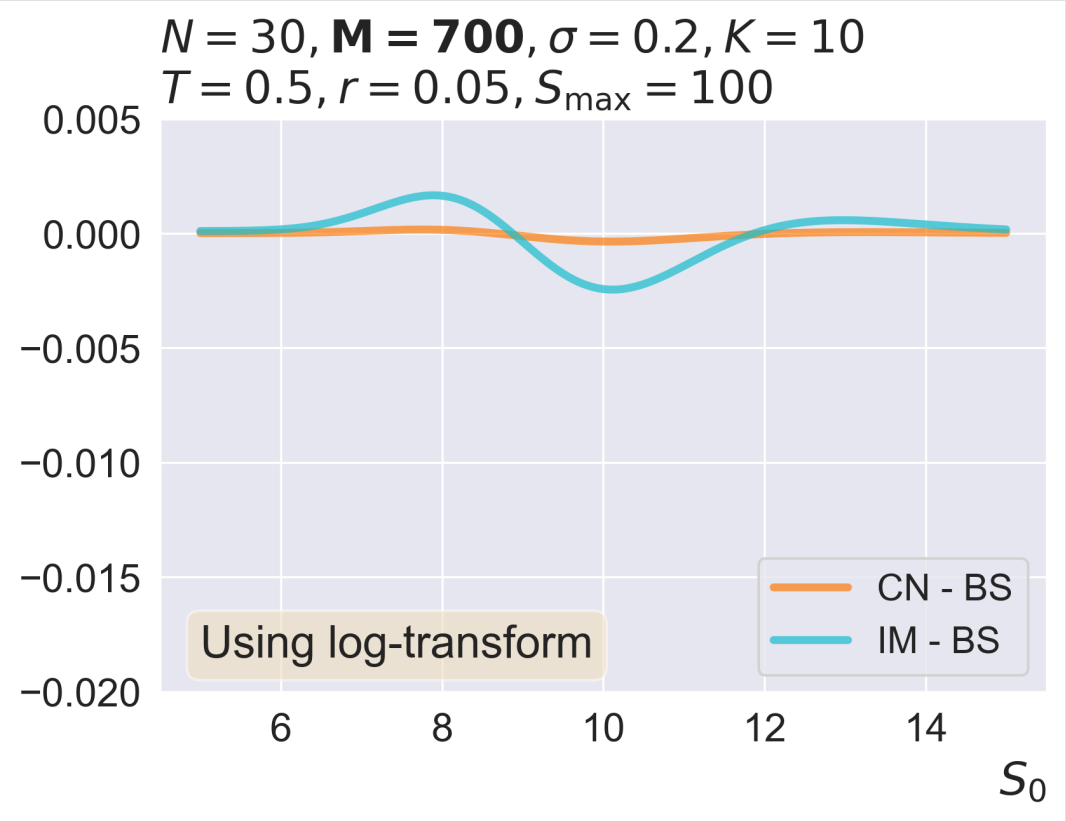


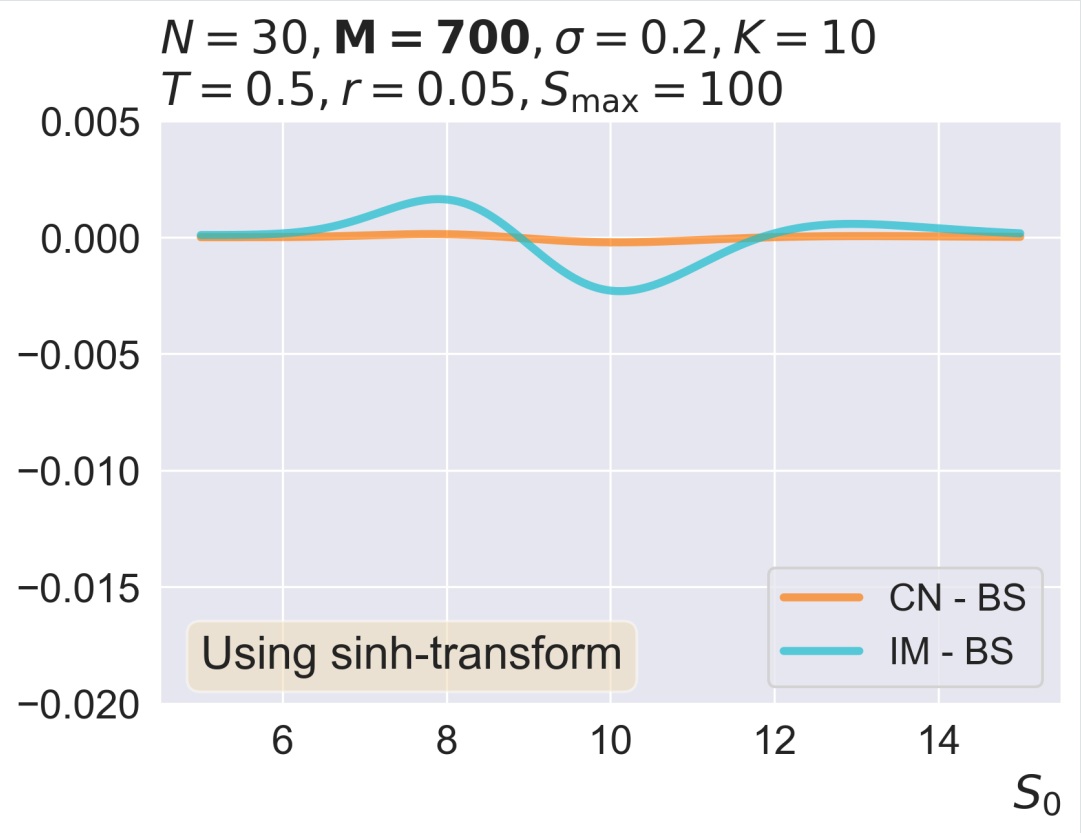
log-transform vs. sinh-transform



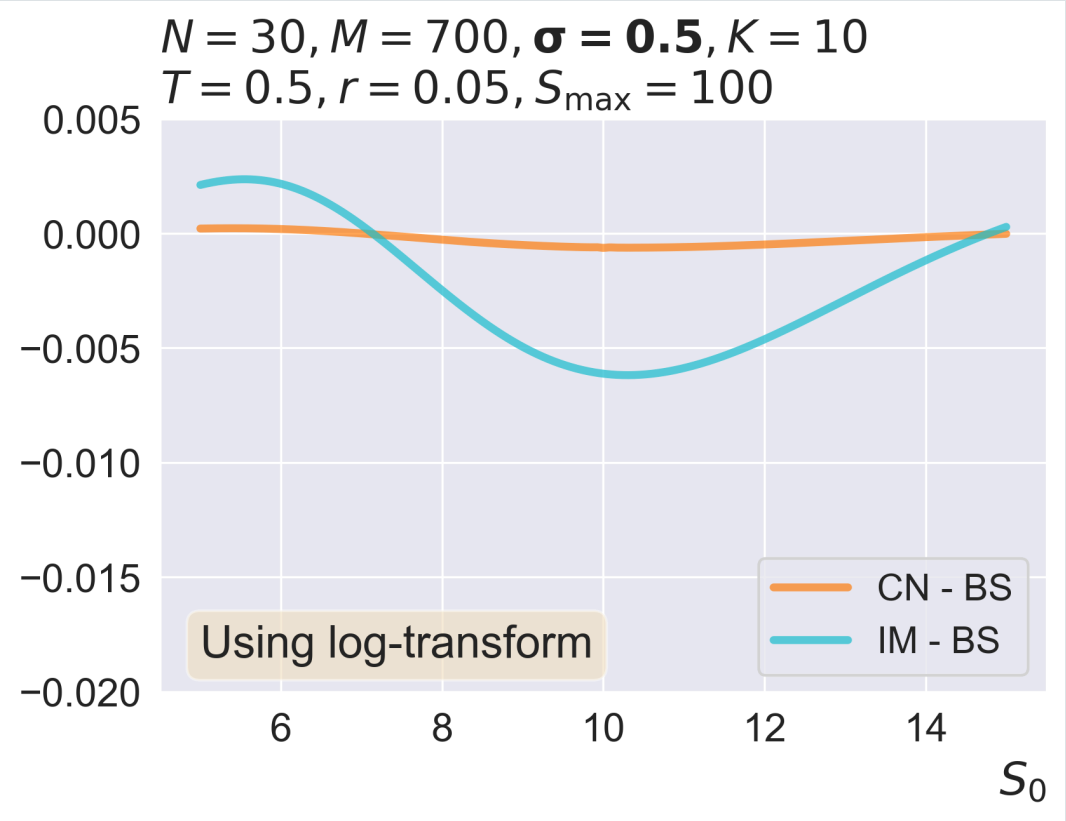


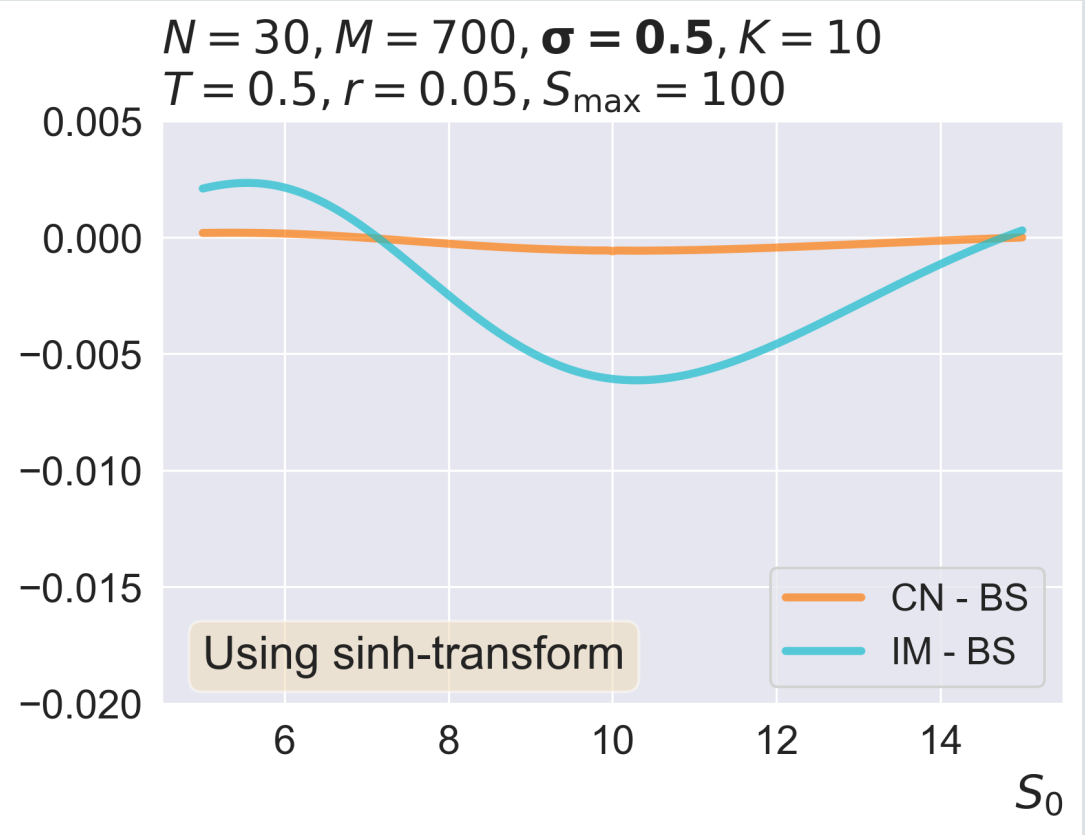
log-transform vs. sinh-transform



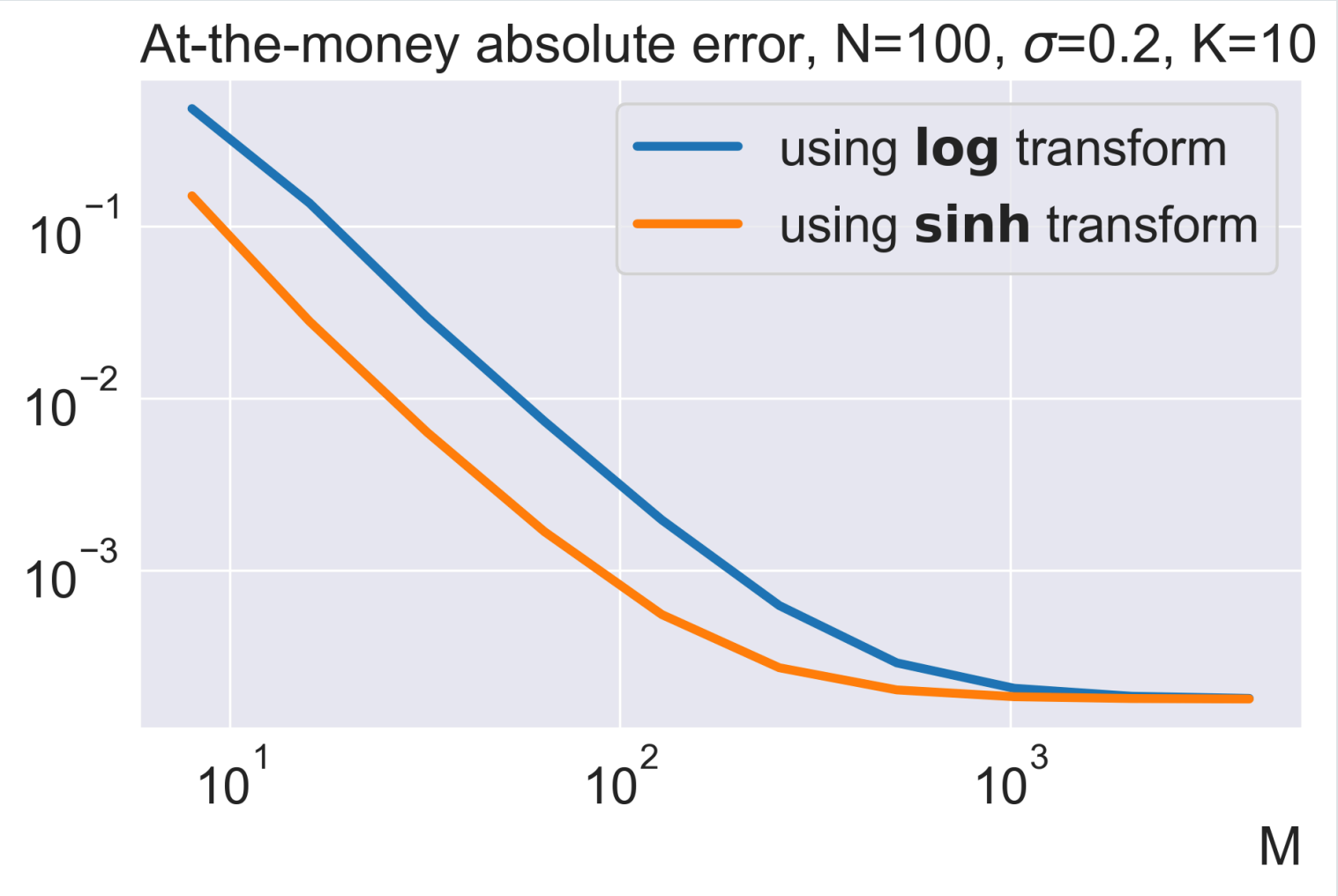


log-transform vs. sinh-transform





log-transform vs. sinh-transform



Conclusions

▷ The explicit scheme is numerically unstable, and should be avoided. The θ-method with $\theta \geq 0.5$ is the finite-difference scheme to use in practice.

▷ Performing a change of variables before applying a finite-difference scheme can greatly improve its accuracy. The best choice of variable transform depends on the problem at hand.

▷ These two conclusions were obtained from numerical experiments with the Black-Scholes PDE, but these are general observations which *remain true* for other types of PDEs.